Data Engineering Pipeline with Airflow, Spark, EzPresto and CUDA-X

Participant lab guide

A hands-on lab on HPE Private Cloud AI that turns 20 million raw viewing events and a live customer database into a validated training table, trains a churn model on the GPU, measures CUDA-X acceleration, and registers the result in MLflow.

Lab 18 · Curriculum 3.1 Feature Engineering & ETL/DataOps · September 2026

Objectives

This lab walks through a complete data-engineering pipeline, from raw files and a live database to a registered model. By the end you will have completed six tasks.

1 Understand the shape of the problem

See why 20 million rows of viewing events and 200,000 customer records cannot train anything on their own, and what has to happen to both before a model can learn from them.

2 Query a live database without copying it

Use EzPresto to explore the `subscribers` table in Postgres from a worksheet, establish the churn base rate, and find which columns carry signal and which carry none.

3 Curate 20 million rows with Spark

Run a Spark job, through Airflow or the Spark Application wizard, that reads 180 daily CSV files, cleans them, and writes day-partitioned Parquet that is 3.66 times smaller.

4 Build and validate a training table on the GPU

Aggregate the events to one row per subscriber with cuDF, join the label from Postgres, and prove every transformation with an executable validation checklist before training.

5 Train, explain and register a model

Train XGBoost on the GPU, read feature importance including a deliberately planted noise column, and register the model under your own name in MLflow.

6 Measure CUDA-X acceleration and state its value

Run the same RandomForest with scikit-learn and cuML, the same nearest-neighbour search with scikit-learn and cuVS, check accuracy before timing, and turn your measured numbers into a presales value statement.

Description

The business question is the one every subscription service asks: **will this subscriber cancel in the next 30 days?** To answer it a model needs one table with one row per person, describing what they did recently and whether they actually cancelled. That table does not exist. It has to be built from two sources that live in different systems, owned by different teams, in different shapes.

The viewing log is 180 daily files of raw events, roughly 100 rows per person, with no outcome attached. The customer records live in a Postgres database, one row per person, and hold the outcome. Neither can train anything alone. The whole data-engineering task is to shrink the first into one row per person and attach the answer from the second, without losing the people who matter most.

One tool appears for each concept, at the moment the problem makes it necessary: Airflow for orchestration, Spark for distributed curation, Parquet for columnar storage, EzPresto for federation, RAPIDS cuDF for GPU acceleration, Kubeflow Notebooks as the workbench, XGBoost for the model, MLflow for the registry, and the CUDA-X libraries cuML and cuVS for accelerated modelling and vector search.

What you will build

- A curated, day-partitioned Parquet dataset of 20,010,929 events, produced by your own Spark job.
- A validated 200,000-row training table built on the GPU, with every transformation checked.
- A churn model with a held-out AUC near 0.87, registered as version 1 under your student number in MLflow.
- Measured CPU-versus-GPU comparisons for aggregation, model training and nearest-neighbour search.

- Three written deliverables: a transformation and validation checklist, a customer-facing data-readiness explanation, and a presales value statement.

Key concepts

These ideas appear throughout the lab. Each is tied to a specific step where you will see it in the data.

Fact and dimension tables

The viewing log is a fact table: many rows per person, each an event, no outcome. The subscriber table is a dimension table: one row per person with attributes and the outcome. Every analytics system is built from these two shapes, and the join between them is where data quietly goes wrong.

Columnar storage and Parquet

CSV stores data row by row as text. Parquet stores it column by column in binary. Repeated values compress well when they sit together, and a query that needs four of eight columns reads only those four. The Spark job in this lab does not shrink the data. It changes the format, and that alone makes it 3.66 times smaller.

Partitions and partition pruning

Spark writes one folder per day, named `dt=2026-01-01` and so on. Reading only the last 30 days means opening 30 folders and never touching the other 150. The speed-up comes purely from how the files were named.

Shuffle

Removing duplicate events forces Spark to redistribute all 20 million rows across executors so that identical keys land together. It is the most expensive stage in the job and the one to look at in the Spark UI.

Federation

EzPresto queries the Postgres database where it lives. No export, no copy, no second system of record. The query engine goes to the data rather than the data coming to the engine.

Feature engineering: the squash

Turning many rows per person into one row per person is a `GROUP BY` in SQL and a `groupby` in pandas or cuDF. The four columns it produces, minutes watched, sessions, completion rate and distinct titles, do not exist anywhere in the source data. You invent them, and that invention is most of applied machine learning.

Absence as information

Subscribers who watched nothing in the last 30 days have no row in the aggregated events. An inner join drops them silently and looks correct. A left join keeps them with their activity filled as zero. In this dataset those silent subscribers churn at more than five times the overall rate. They are the strongest signal there is.

Hardware acceleration with the same API

cuDF mirrors the pandas API, cuML mirrors scikit-learn, and XGBoost moves to the GPU with a single keyword. The lab runs identical code on CPU and GPU, checks the results agree, and only then compares the clock.

Labels and supervised learning

A model learns from examples where the outcome is already known. The column `churned_next_30d` is that outcome. Past exam papers with the answer key are training data. The real exam has no key, and that is prediction.

Model registry

A trained model in a notebook is invisible to everyone else. MLflow records what was trained, with which parameters, by whom, and what score it got, and makes it findable, comparable, auditable and deployable.

Platform services

Nine components carry out the lab. Each owns one stage and hands a versioned artifact to the next.

Table 1. Platform services and what each one owns

Component	Role in the lab	Touches the data?
Airflow	Runs the Spark stage once per participant from one parameterised DAG	Never; it only tells Spark to run
Spark Operator	Executes the curate job as a SparkApplication on Kubernetes	Reads and writes 20 million rows
Shared data volume	Holds the raw CSV and the curated Parquet, visible to Spark and to notebooks	Stores it
Postgres	Holds the `subscribers` table with the churn label	Stores it
EzPresto	Federated SQL over Postgres from a worksheet, no copy	Reads
Kubeflow Notebooks	The GPU workbench where the training notebook runs	Hosts every step below
RAPIDS cuDF, cuML, cuVS	GPU dataframes, GPU scikit-learn, GPU vector search	Reads 20 million rows, produces 200,000
XGBoost	Gradient-boosted trees on the GPU	Reads 200,000 rows
MLflow	Experiment tracking and the model registry	Stores the model

Lab environment

Each participant works in their own project namespace on a shared HPE Private Cloud AI cluster, with one GPU allocated to their notebook server. Raw data is a single shared read-only copy. Only outputs are per participant, keyed by a zero-padded student number such as `student-07`.

Platform	HPE Private Cloud AI, AI Essentials 1.9
Platform domain	ai-application.pcai1.genai1.hou
Notebook GPU	One NVIDIA L40S (46 GB) per participant notebook server
Raw data	20,010,929 viewing events, 180 daily CSV files, 849 MB, on the shared volume under `raw/watch_events/`
Label source	Postgres `app_db.public.subscribers`, 200,000 rows, EzPresto catalog `dataengineeringlab`
Spark	Spark 3.5.5 via the PCAI Spark Operator, 1 driver core and 4 GB, 1 executor core and 4 GB
Notebook packages	cudf, rmm, cuml, cuvs 26.08 (CUDA 12) · numpy 2.4 · pandas 3.0 · scikit-learn 1.9 · xgboost · psycopg2 · mlflow 2.22
Model registry	MLflow, reached from the notebook at `http://mlflow.mlflow.svc.cluster.local:5000`

Note

The dataset is synthetic and generated with a fixed seed, so every number quoted in this guide is reproducible exactly. It is not a public dataset, and the churn label was constructed to depend causally on the viewing behaviour, which is what makes the model learnable.

Before you begin

1. **Know your student number.** Every path, experiment and model name in the lab derives from it. Use it consistently, zero-padded where the lab asks for it.
2. **Log in to HPE AI Essentials** at `https://home.ai-application.pcai1.genai1.hou/` with your own account. Every step runs under your identity.
3. **Have a GPU notebook server in your own project.** Open Notebooks, select your project in the Project selector at the top of the page, and confirm the notebook server assigned to your student number is Running with one GPU. A notebook created in another project will fail at the MLflow step, because the platform provisions the MLflow login token only in your own project.
4. **Confirm the EzPresto data source exists.** Under Data Engineering, Data Sources, a PostgreSQL source named `dataengineeringlab` should show as Connected. If it does not, tell the instructor.

Pipeline overview

The lab runs as a pipeline. Each stage produces an artifact the next stage consumes, and the row count changes at every step.

1 Stage 1 - Look at the raw data

Inspect the 180 daily folders on the shared volume and confirm why a raw event log cannot be trained on: many rows per person, no outcome.

2 Stage 2 - Explore the label with EzPresto

Query `subscribers` through the federated catalog. Establish the base rate, then test plan, support tickets, tenure, payment method and country one by one. Country is planted noise, and the queries show how noise looks.

3 Stage 3 - Curate the events with Spark

Open Airflow from Tools & Frameworks and trigger the `churn_pipeline` DAG, entering your student number in the trigger form. It submits a Spark job that reads the CSV with an explicit schema, removes duplicates, filters bad rows and writes day-partitioned Parquet into your own curated folder. Same 20,010,929 rows in and out, 3.66 times smaller.

4 Stage 4 - Build the training table on the GPU

In the notebook, read only the last 30 day folders, aggregate 20 million events to one row per subscriber with cuDF, fetch the label from Postgres, join the two halves on the GPU with a left join, and run the validation checklist.

5 Stage 5 - Train, explain, register

Train XGBoost on the GPU, inspect feature importance, and register the model in MLflow as `student-NN-churn`. Compare your AUC with the room.

6 Stage 6 - Measure CUDA-X and write the value

Run cuML against scikit-learn and cuVS against scikit-learn brute force, accuracy first and then the clock, collect every number you measured, and write three value statements.

Lab sections

The notebook is organised into eleven parts plus one validation sub-part. Stages 1 to 3 happen outside the notebook.

Table 2. Lab sections and where each one runs

Section	Where	Responsibility
Stage 1 - Raw data	Notebook terminal	See the 180 daily files and the shape of the problem
Stage 2 - EzPresto	Query Editor	Nine queries that build to the country punchline
Stage 3 - Spark	Airflow (Tools & Frameworks), or the Spark wizard	Curate CSV to Parquet under your student number
Part 1 - Setup	Notebook	Environment check, student number, GPU check
Part 2 - What Spark produced	Notebook	Parquet, partitions, partition pruning

Section	Where	Responsibility
Part 3 - The squash	Notebook	Same aggregation on CPU then GPU, timed
Part 4 - The label	Notebook	Read `subscribers` from Postgres
Part 5 - The join	Notebook	Left join on the GPU, zeros for silence
Part 5b - Validation	Notebook	Executable checklist, must print ALL PASS
Part 6 - Train	Notebook	XGBoost on the GPU, held-out AUC
Part 7 - Importance	Notebook	What the model used, and the noise fingerprint
Part 8 - Register	Notebook	Log and register in MLflow
Part 9 - cuML	Notebook	RandomForest, scikit-learn versus cuML
Part 10 - cuVS	Notebook	Look-alike search, scikit-learn versus cuVS
Part 11 - Your numbers	Notebook	Measured-results table for the value statements

Lab walkthrough

Work through the stages in order. Each step shows what to run and the output you should see.

Stage 1 - Look at the raw data

Open a terminal in your notebook server and list the raw data on the shared volume.

```
ls ~/shared/data-engineering-lab/raw/watch_events | head -5
ls ~/shared/data-engineering-lab/raw/watch_events | wc -l
head -3 ~/shared/data-engineering-lab/raw/watch_events/dt=2026-06-15/events.csv
```

You should see folders named by date, 180 of them, and inside each a single `events.csv`. The first rows look like this.

```
event_id,subscriber_id,content_id,event_date,watch_minutes,device,completed
17456650,3,1188,2026-06-15,22.4,tv,1
17456651,4,347,2026-06-15,8.1,mobile,0
```

Every row is one viewing session. There is no column saying whether the subscriber cancelled, and there are roughly 100 rows per person. This is the fact table. Nothing here can be trained on until it is one row per person with the outcome attached.

Stage 2 - Explore the label with EzPresto

EzPresto is the federated query engine behind the platform's Data Catalog. Its tile under **Tools & Frameworks** shows it Ready, and its Open button leads to the engine's own cluster console. For this lab you work through the Data Catalog and Query Editor in the platform instead, which query the same engine.

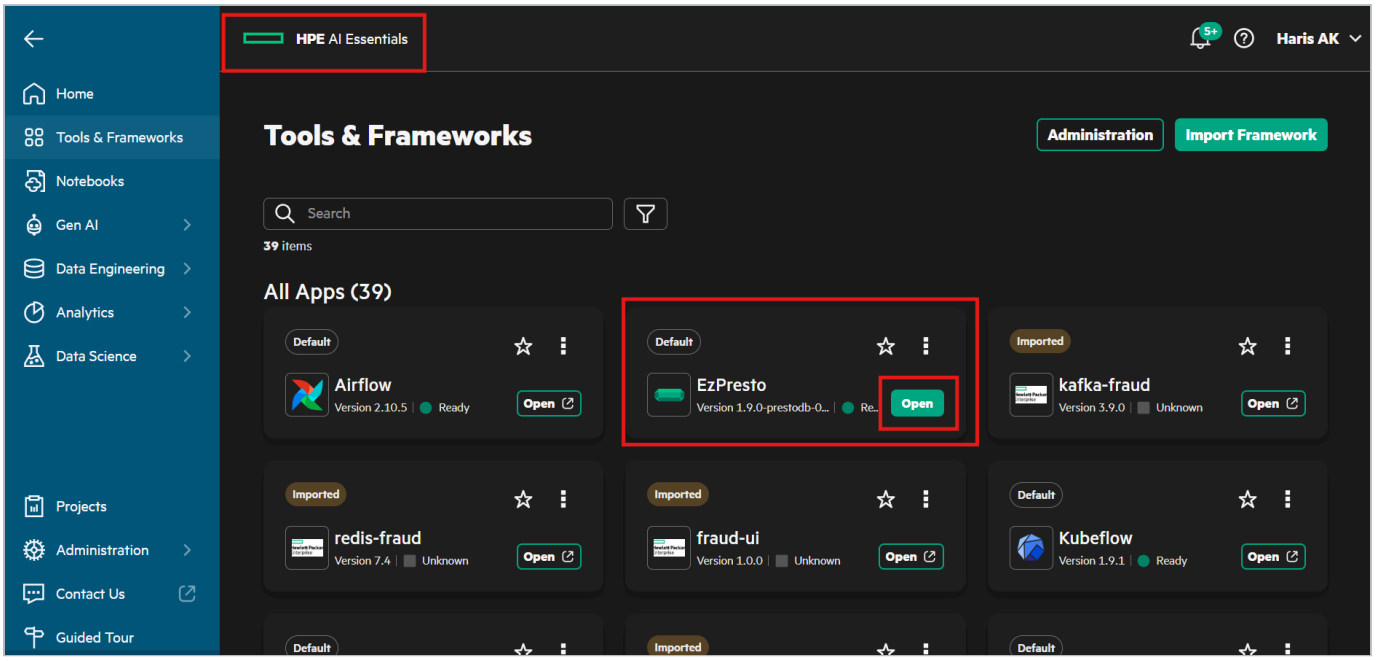

Figure 1. The EzPresto tile under Tools & Frameworks.

The PostgreSQL source `dataengineeringlab` was registered by the instructor and shows as Connected under **Data Engineering, Data Sources**. Nobody in the lab has a database password or client. The catalog is the only way in, and it is read-only.

Step 2.1 - Select the table in the Data Catalog

1. In the left navigation choose **Data Engineering**, then **Data Catalog**.

2. Under **Connected Data Sources**, expand `dataengineeringlab` and tick the `public` schema.
3. The `subscribers` table appears under **All Datasets**. Click **Select** on its row. The button changes to Deselect and the **Selected Datasets** counter at the top right shows 1.

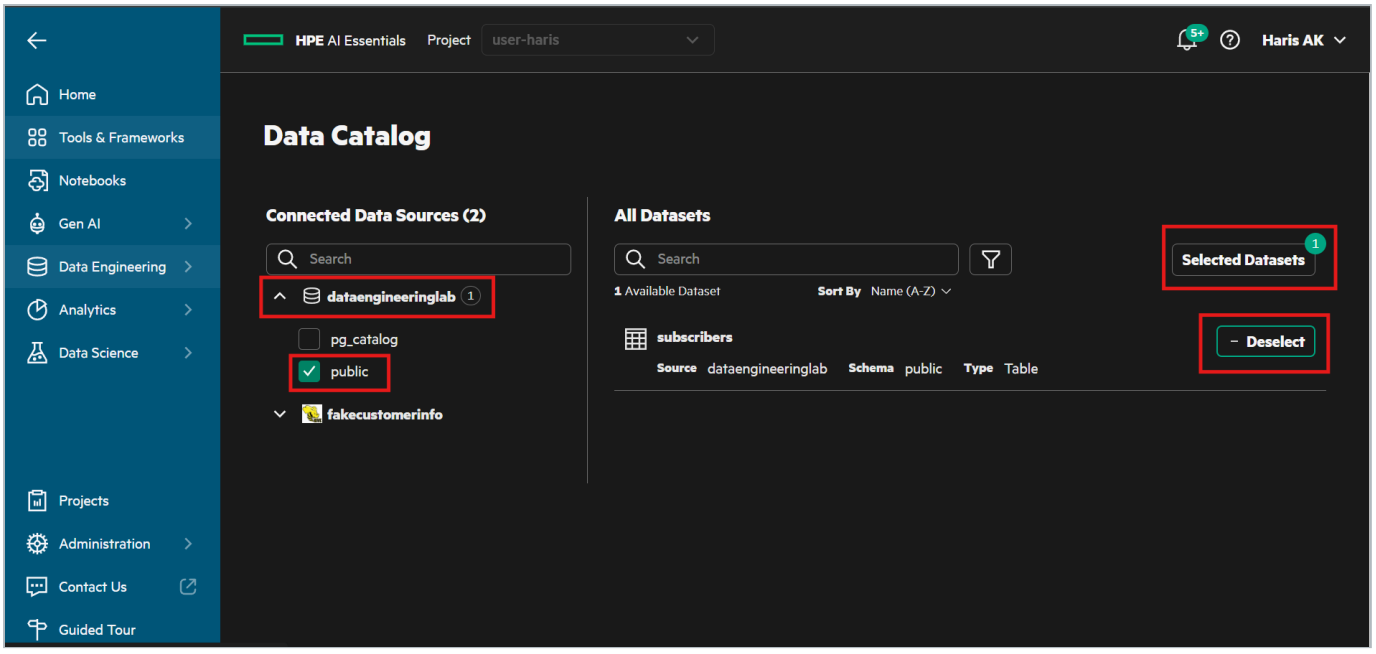

Figure 2. Data Catalog. Expand dataengineeringlab, tick public, and select the subscribers table.

1. Click **Selected Datasets**. A panel opens listing the table with its source, schema and type.
2. Click **Query Editor** in that panel.

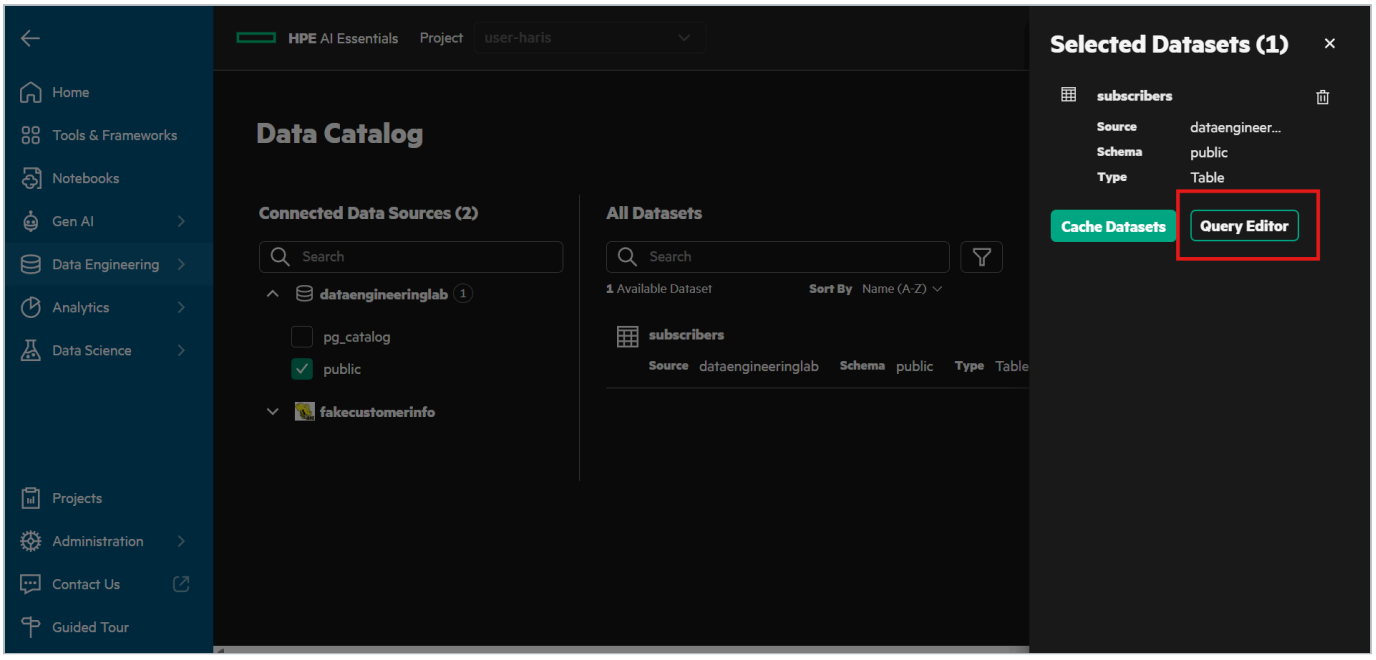

Figure 3. The Selected Datasets panel. Query Editor opens a worksheet on the selected table.

Step 2.2 - Run a query

The Query Editor opens with the table listed under Selected Datasets on the left and an empty worksheet on the right. Type or paste a statement into the **SQL Query** box, leave **Run Options** at Preview, and click **Run**.

Important

The worksheet accepts one statement at a time and rejects a trailing semicolon. It also defaults to a preview limit of 1,000 rows. Aggregates are unaffected, but if a plain `SELECT` seems to stop at 1,000 rows, the limit is why.

```
SELECT count(*) AS subscribers,
       sum(churned_next_30d) AS churned,
       round(avg(churned_next_30d), 4) AS churn_rate
FROM   dataengineeringlab.public.subscribers
```

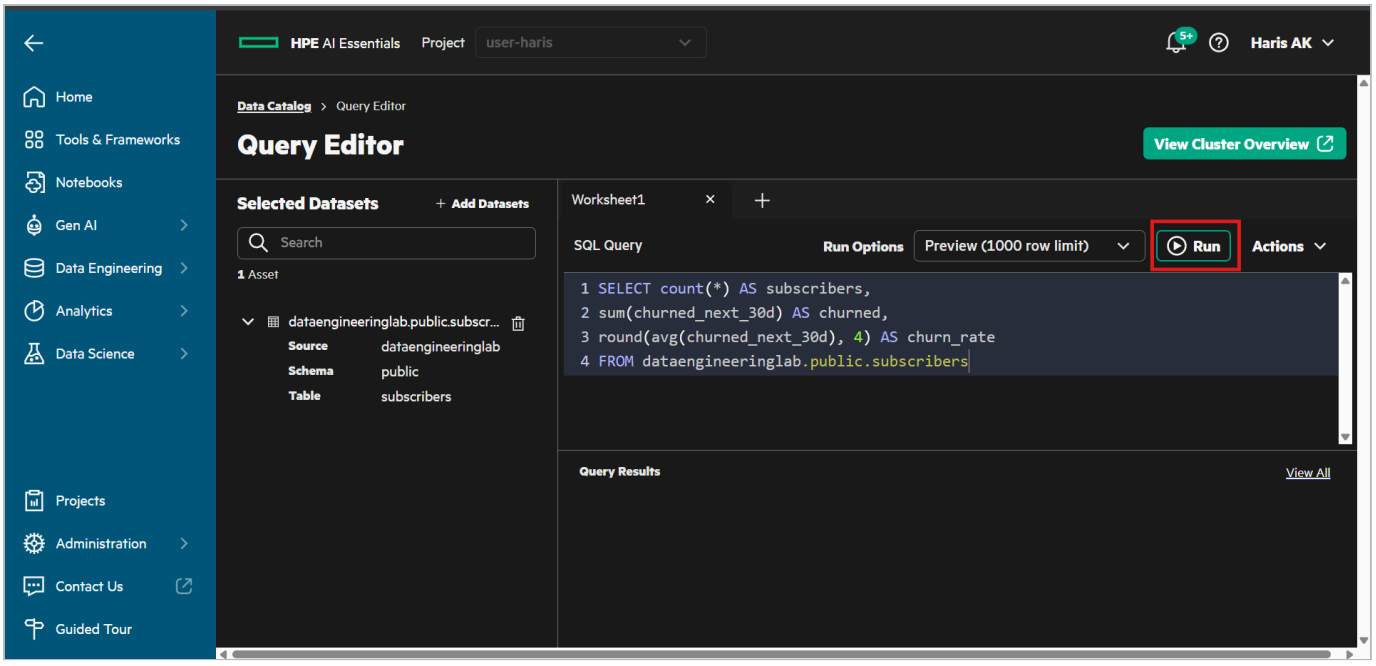

Figure 4. The Query Editor with the base-rate query entered. Run executes it.

The query appears under **Query Results** with a Status of Succeeded and its duration. Click the arrow at the left of the row to expand the result rows, or **View All** for the full history.

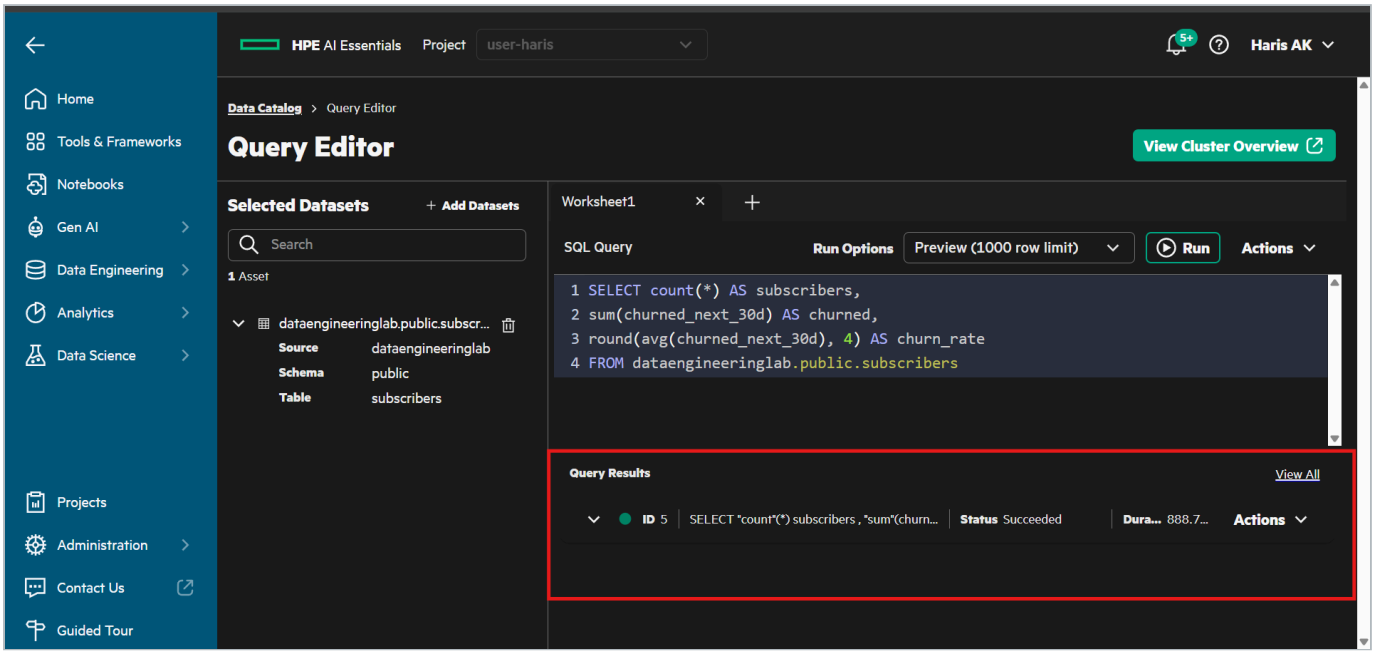

Figure 5. Query Results. Each statement you run is listed with its status; expand a row to see the values.

Expected: 200000 subscribers, 23796 churned, churn rate 0.119. Roughly 12 percent is the base rate, and every later number is only meaningful compared with it. Run each of the following statements the same way.

Step 2.3 - Does the plan matter?

```
SELECT plan, count(*) AS people, round(avg(churned_next_30d), 4) AS churn_rate
FROM   dataengineeringlab.public.subscribers
GROUP  BY plan ORDER BY churn_rate DESC
```

Table 3. Churn rate by plan

plan	people	churn_rate
basic	89977	0.1526
standard	70123	0.0973
premium	39900	0.0812

Basic churns nearly twice as often as premium. Real signal.

Step 2.4 - Do support tickets matter?

```
SELECT support_tickets_90d, count(*) AS people, round(avg(churned_next_30d), 4) AS churn_rate
FROM   dataengineeringlab.public.subscribers
GROUP  BY support_tickets_90d HAVING count(*) > 200
ORDER  BY support_tickets_90d
```

Table 4. Churn rate by support tickets in the last 90 days

tickets	people	churn_rate
0	141001	0.0986
1	49185	0.1536
2	8635	0.2249
3	1069	0.3265

Every complaint roughly compounds the risk. This is the clearest relationship in the data.

Step 2.5 - Does tenure matter?

```
SELECT CASE WHEN tenure_months <=  6 THEN '1. 0-6 mo'
            WHEN tenure_months <= 12 THEN '2. 7-12 mo'
            WHEN tenure_months <= 24 THEN '3. 13-24 mo'
            WHEN tenure_months <= 36 THEN '4. 25-36 mo'
            ELSE '5. 37+ mo' END AS tenure_band,
       count(*) AS people, round(avg(churned_next_30d), 4) AS churn_rate
FROM   dataengineeringlab.public.subscribers
GROUP  BY 1 ORDER BY 1
```

Table 5. Churn rate by tenure band

tenure band	people	churn_rate
0-6 months	20213	0.2070
7-12 months	20597	0.1826
13-24 months	40506	0.1484
25-36 months	41077	0.1097
37+ months	77607	0.0687

New customers churn three times more than long-standing ones, and the relationship is monotonic. Bucketing a number into bands is a real analyst move: month by month is noise, bands show the shape.

Step 2.6 - Does country matter?

```
SELECT country, count(*) AS people, round(avg(churned_next_30d), 4) AS churn_rate
FROM   dataengineeringlab.public.subscribers
GROUP  BY country ORDER BY churn_rate DESC
```

Table 6. Churn rate by country

country	people	churn_rate
DE	24148	0.1211
US	60091	0.1197
IN	49988	0.1191
BR	29947	0.1184
JP	15838	0.1172
GB	19988	0.1162

Six countries, every one within half a percent of the base rate. Country predicts nothing. It was placed in the data as deliberate noise. Germany looks worse than Britain only until you notice that every other column varies by two to three times and this one varies by half a percent.

Step 2.7 - Prove it with one number

```
SELECT round(max(r) - min(r), 4) AS spread
FROM  (SELECT avg(churned_next_30d) AS r
       FROM   dataengineeringlab.public.subscribers
       GROUP  BY country)
```

Expected spread for country: 0.0049. Change `GROUP BY country` to `GROUP BY plan` and the spread becomes 0.0715, fifteen times wider. A useless column is not one that scores zero. It is one whose categories are indistinguishable from each other. The same fingerprint reappears in the model's feature importance in Part 7.

Step 2.8 - Two sources in one statement

```
SELECT (SELECT count(*) FROM dataengineeringlab.public.subscribers) AS subscribers_in_postgres,
       (SELECT count(*) FROM system.runtime.nodes)                  AS query_engine_nodes
```

The first catalog is a Postgres database on another machine. The second is the query engine's own internal catalog. Two different systems, one `SELECT`, no copy of either. That is federation, and swapping `system` for a second real database changes nothing about how the query is written.

Stage 3 - Curate the events with Spark

The Spark job reads the 180 raw CSV files with an explicit schema, drops duplicate events, keeps only rows with a subscriber and a plausible duration, adds an `is_long_view` flag, and writes Parquet partitioned by day into your own curated folder. It is submitted through Airflow. The Spark Application wizard is the fallback and produces identical output.

Step 3.1 - Open Airflow

In the left navigation choose **Tools & Frameworks**. Find the **Airflow** tile and click **Open**. Airflow opens in a new browser tab.

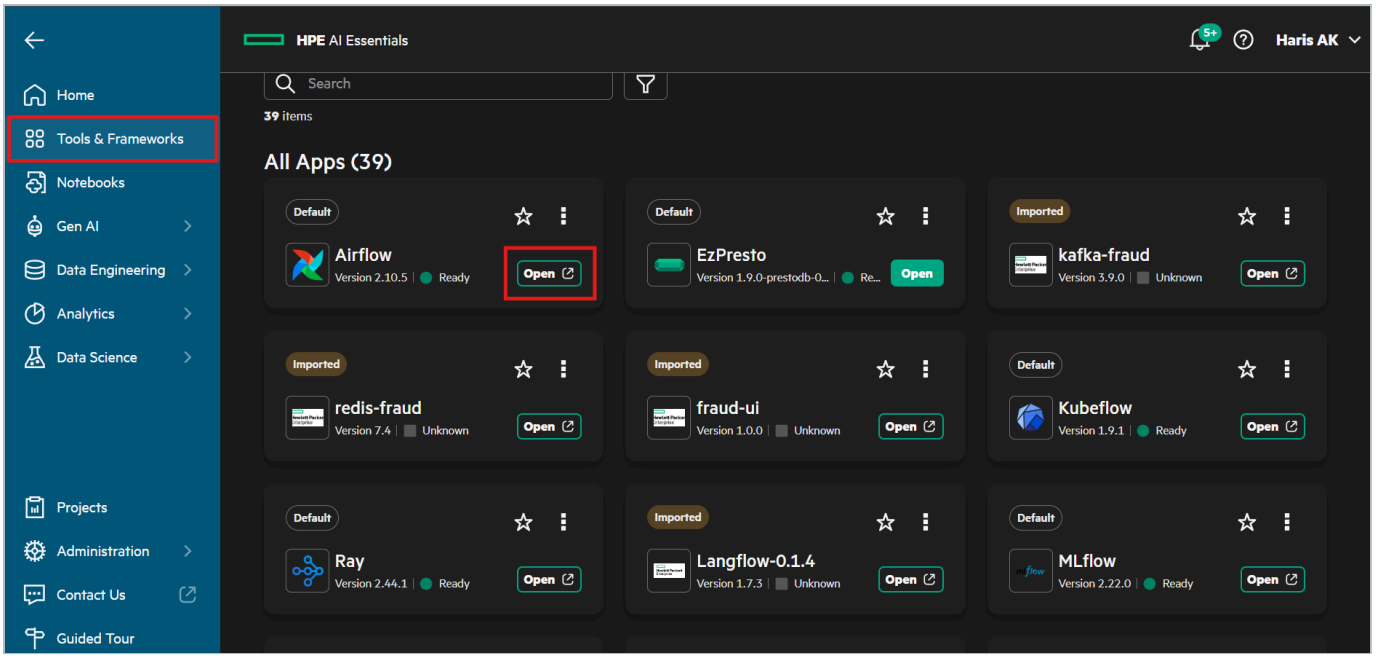

Figure 6. Tools & Frameworks. The Airflow tile and its Open button.

Step 3.2 - Trigger the DAG

The DAG list shows one DAG, `churn_pipeline`, tagged churn, lab and spark.

1. Check that the toggle to the left of the DAG name is on. A paused DAG queues a trigger and never runs it.
2. Click the play button under **Actions** at the right of the row. A trigger form opens.
3. In the **Student number** field enter your student number without leading zeros, for example `7`, and click **Trigger**.

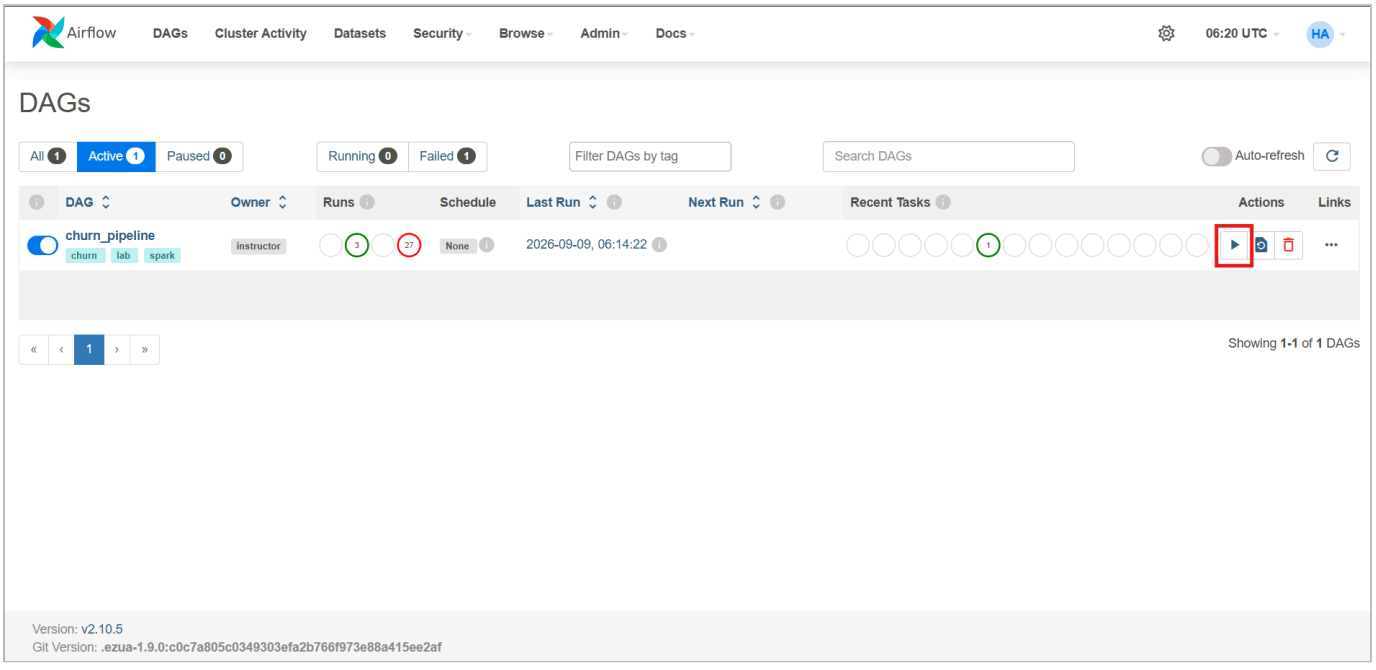

Figure 7. The Airflow DAG list. The churn_pipeline toggle is on, and the play button under Actions opens the trigger form.

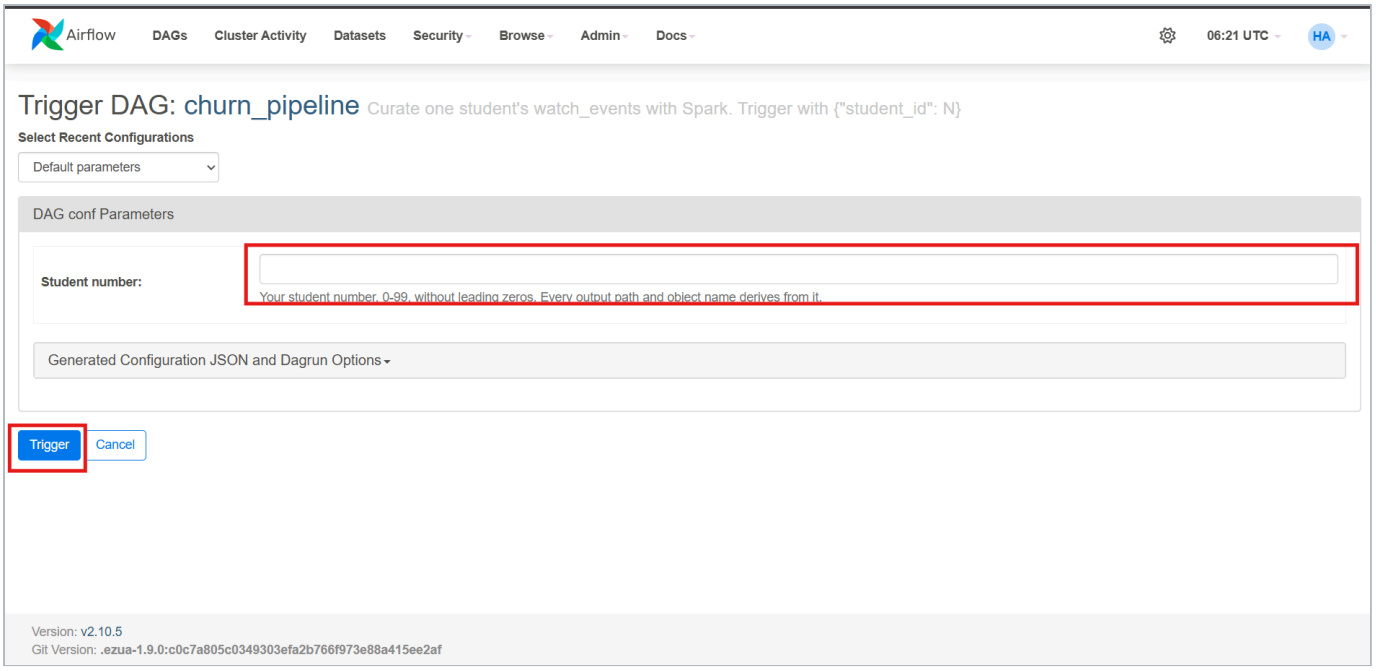

Figure 8. The trigger form. Enter your student number in the Student number field and click Trigger.

Every path and object name the job uses derives from that number, so thirty participants produce thirty independent runs of the same DAG and nobody edits a file. Leaving the field empty fails immediately with the message `No student_id supplied`. Trigger again with it filled in.

Step 3.3 - Watch the run

Click the DAG name to open it. The **Grid** view shows one column per run, newest on the right. Click the newest bar to see the run details: the Status reads success when it is done, and the run duration is about two and a half minutes. Click the `curate` square below the bar and open the **Logs** tab to read the task log, which shows the SparkApplication being submitted, its state changing about every ten seconds, and ends with the driver log's own summary.

```
[dag] submitted curate-student-07
[dag]   reads  file:///mounts/shared-volume/shared/data-engineering-lab/raw/watch_events
[dag]   writes file:///mounts/shared-volume/shared/data-engineering-lab/curated/student-
07/watch_events
[dag] 12:52:55  SUBMITTED
[dag] 12:53:05  RUNNING
[dag] 12:55:05  COMPLETED
[dag] ---- driver log (last 40 lines) ----
[curate] input partitions: 180  (one per daily file — this is your parallelism)
============================================================
[curate] rows written    : 20,010,929
[curate] subscribers     : 200,000
[curate] day partitions  : 180
============================================================
```

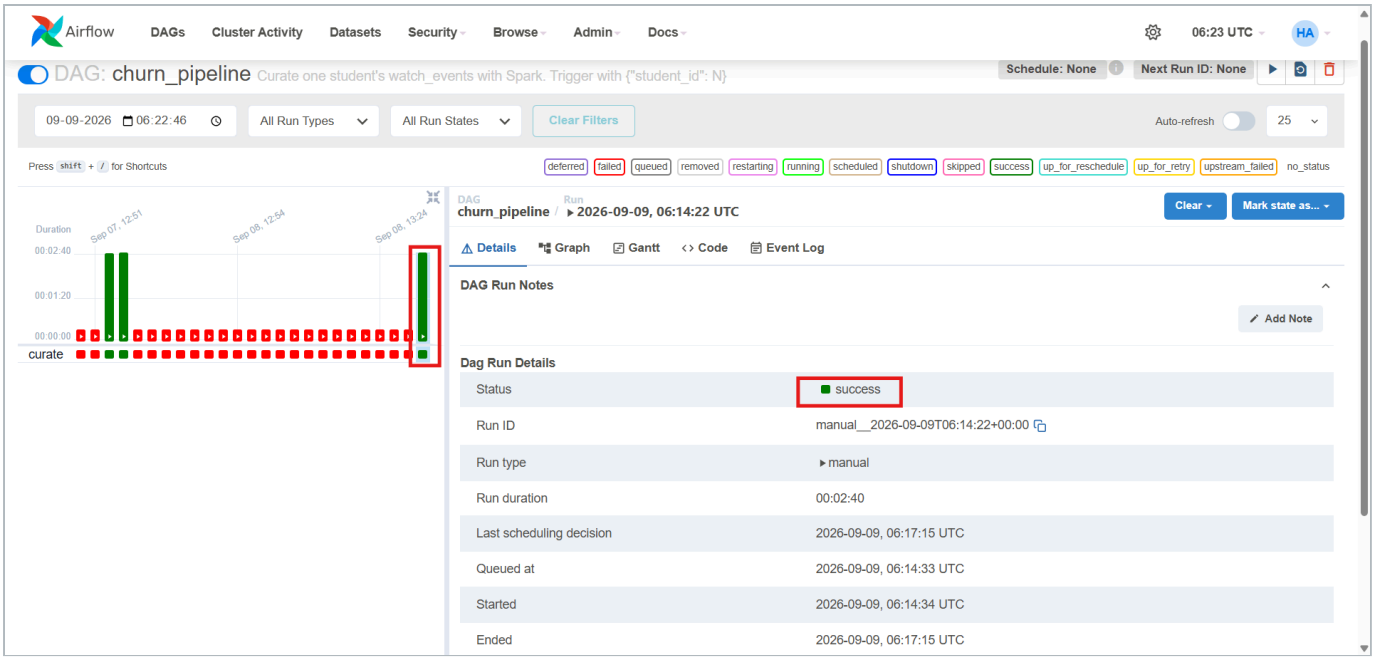

Figure 9. The Grid view after a successful run. The newest run is selected and its status is success.

The run takes about two and a half minutes. The Spark application it created is also visible in the platform: in the left navigation choose **Analytics**, then **Spark Applications**.

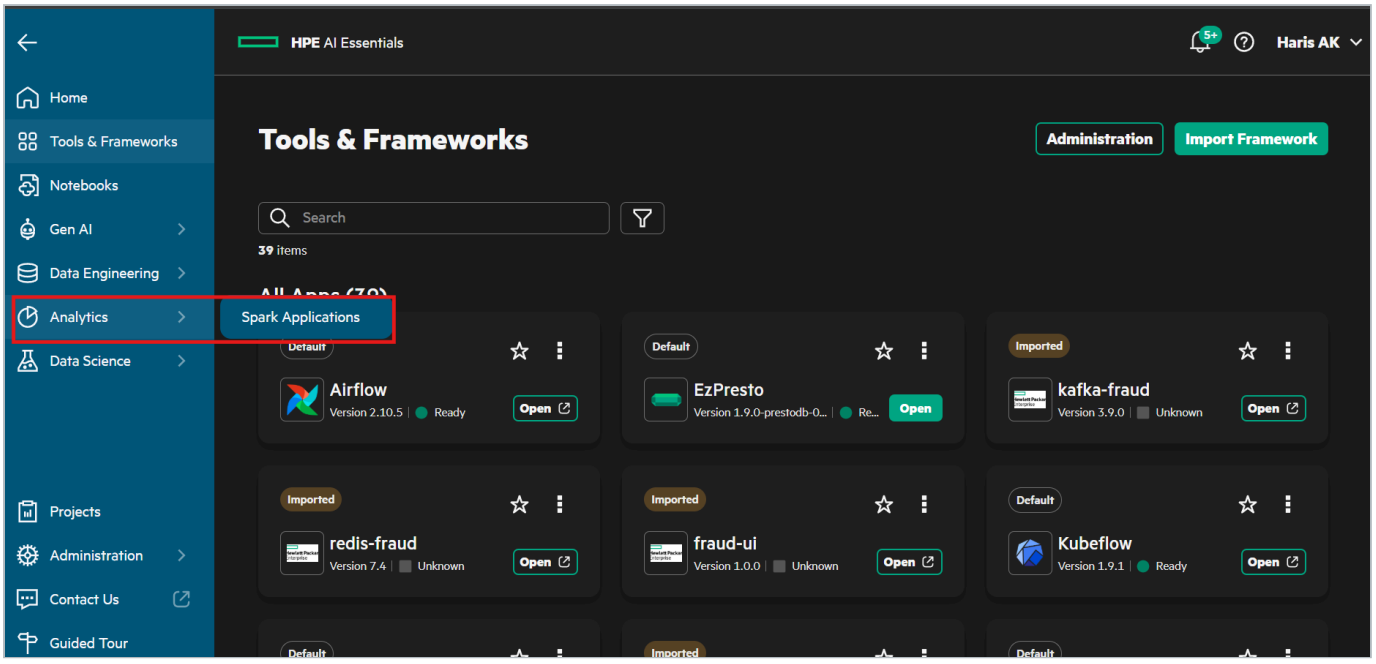

Figure 10. Analytics, Spark Applications in the left navigation.

Find the row named with your student number, `curate-student-NN`. Its status should read Completed with a duration of about 1 minute 50 seconds. Every participant's run appears in this list, so check the name before reading the status.

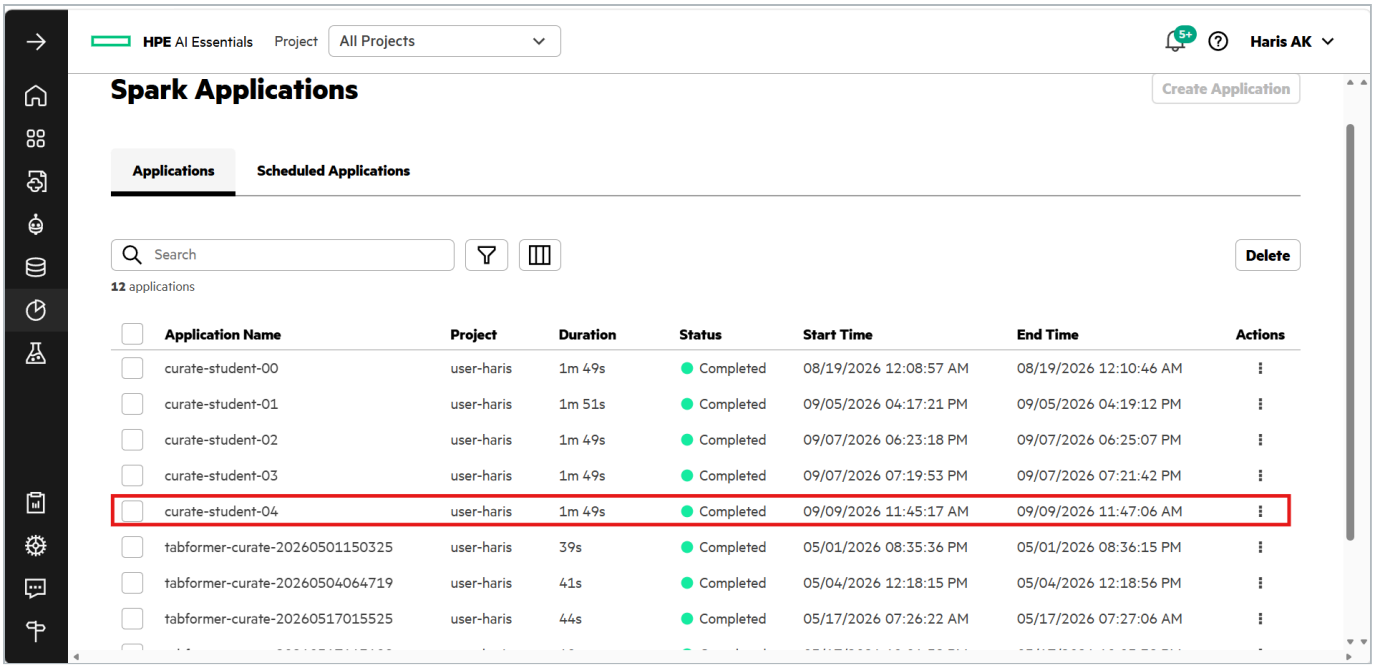

Figure 11. The Spark Applications list. The application Airflow created for one participant, curate-student-04, is Completed after 1 minute 49 seconds.

Step 3.4 - Verify the output

In the JupyterLab file browser of your notebook server, open **shared**, then **data-engineering-lab**, then **curated**. A folder named with your student number is there, next to the folders of the other participants.

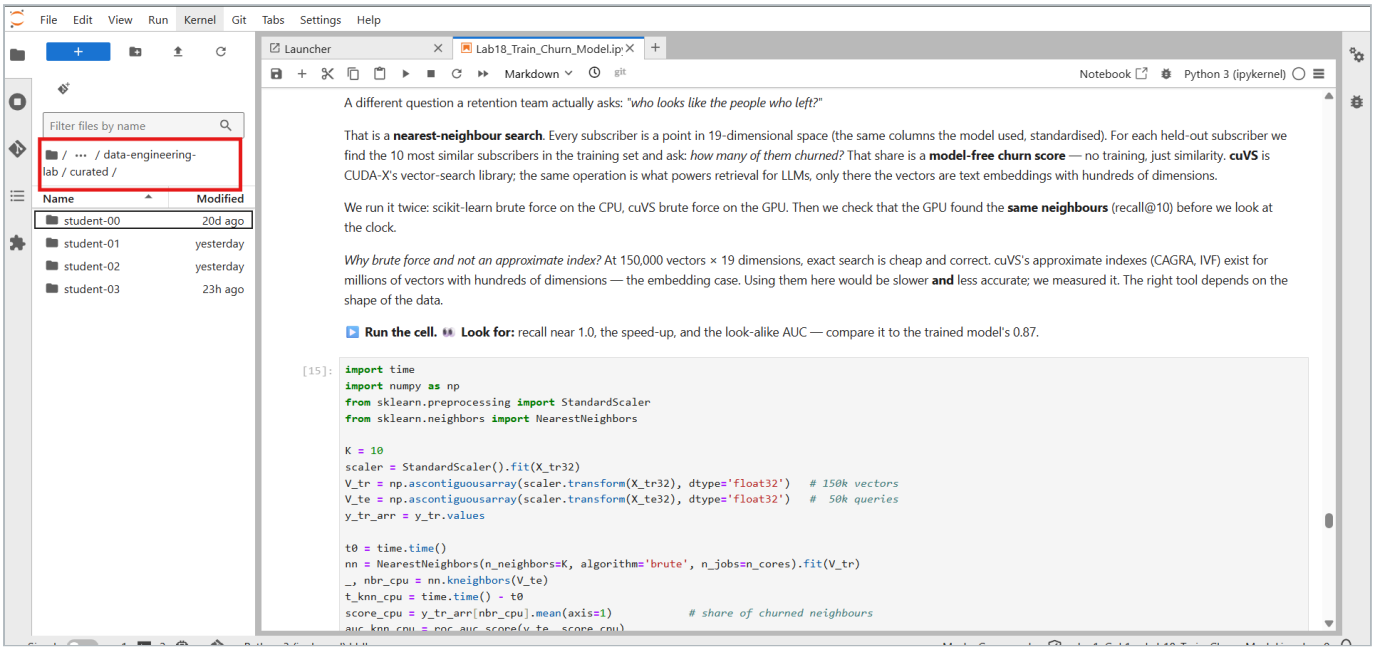

Figure 12. The curated folder on the shared volume, one sub-folder per participant.

Then, from a terminal in the same notebook server, count the partitions and the size of what Spark wrote. Replace `07` with your number.

```
python3 -c "import glob,os; o=os.path.expanduser('~/shared/data-engineering-lab/curated/student-07/watch_events'); print(len(glob.glob(o+'/dt=*')),'partitions,',
round(sum(os.path.getsize(os.path.join(r,f)) for r,_,fs in os.walk(o) for f in fs)/1e6),'MB')"
```

Expected: `180 partitions, 232 MB`. The raw CSV was 849 MB, so the Parquet copy is 3.66 times smaller while holding the same 20,010,929 rows. Nothing was dropped and nothing was aggregated. Only the format changed.

Fallback - the Spark Application wizard

If Airflow is unavailable, submit the same job by hand under **Analytics, Spark Applications, Create Spark Application**. Leave scheduling off so the job runs once.

Table 7. Spark Application wizard settings

Screen	Setting
Application Details	Name `curate-student-NN`
Configure Spark Application	Type **Python**, Source **Shared Folder**, Class Name empty. File Name: Browse to `data-engineering-lab/spark/curate_events.py`
Arguments, in order	`NN` · `file:///mounts/shared-volume/shared/data-engineering-lab/raw/watch_events` · `file:///mounts/shared-volume/shared/data-engineering-lab/curated/student-NN/watch_events`
Dependencies	None
Driver	1 core, 4G
Executor	1 instance, 1 core, 4G
Schedule	Off, so the application runs once

Note

The application file uses the `local://` scheme because it is already on the pod. The two data arguments use `file://` because they are filesystem paths. Browse fills the first one in for you.

Stage 4 - Build the training table on the GPU

Step 4.1 - Open your notebook server

In the left navigation choose **Notebooks**. In the **Project** selector at the top of the page choose your own project. The Notebook Servers list shows the server assigned to your student number with one GPU in the GPUs column and a status of Running. Click its name to connect.

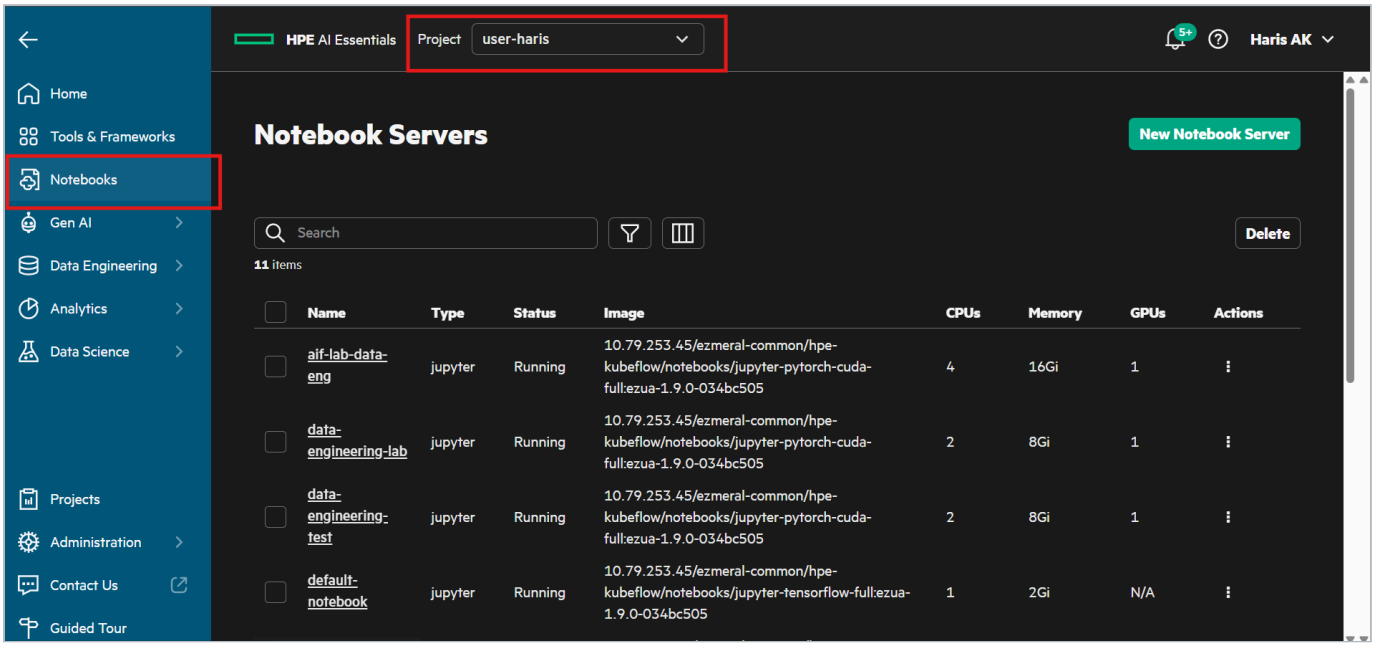

Figure 13. The Notebooks page. Select your project at the top, then connect to the notebook server assigned to you.

Step 4.2 - Open the lab notebook

JupyterLab opens on the Launcher. In the file browser on the left, open the folder `pcai-dataeng-lab`, then the folder `notebooks`, and double-click `Lab18_Train_Churn_Model.ipynb`.

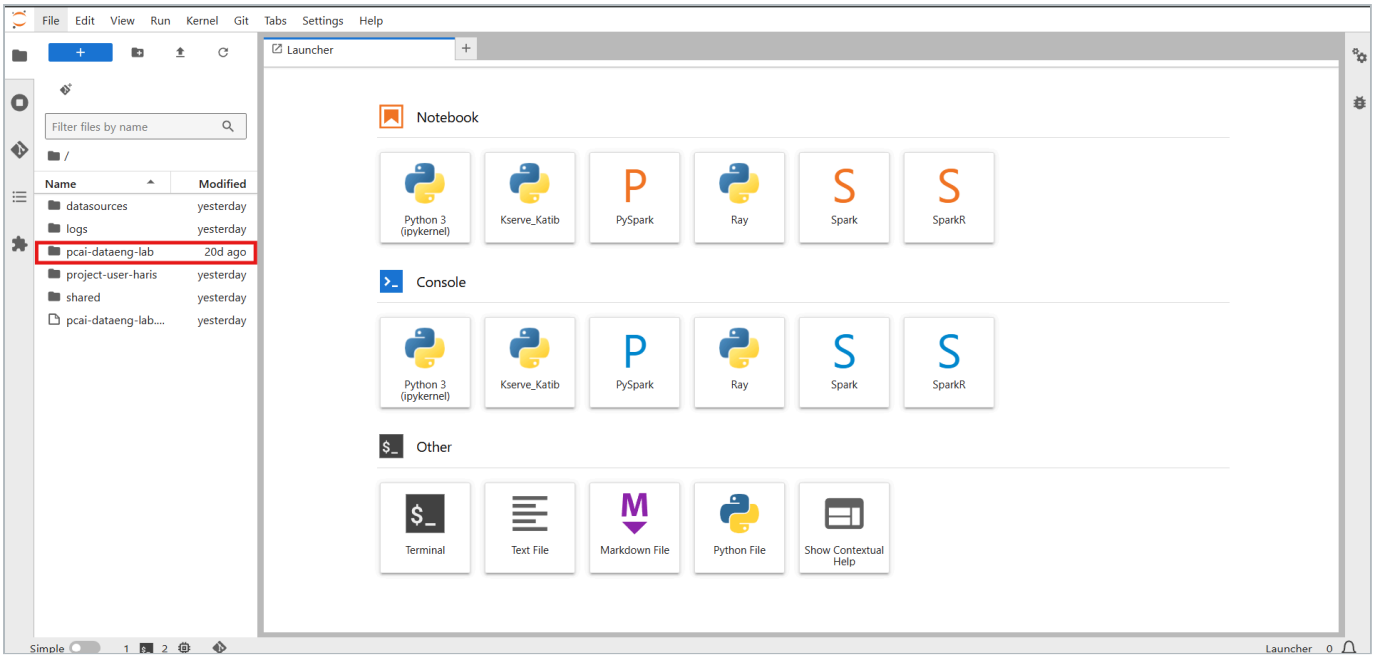

Figure 14. The JupyterLab file browser. Open the pcai-dataeng-lab folder.

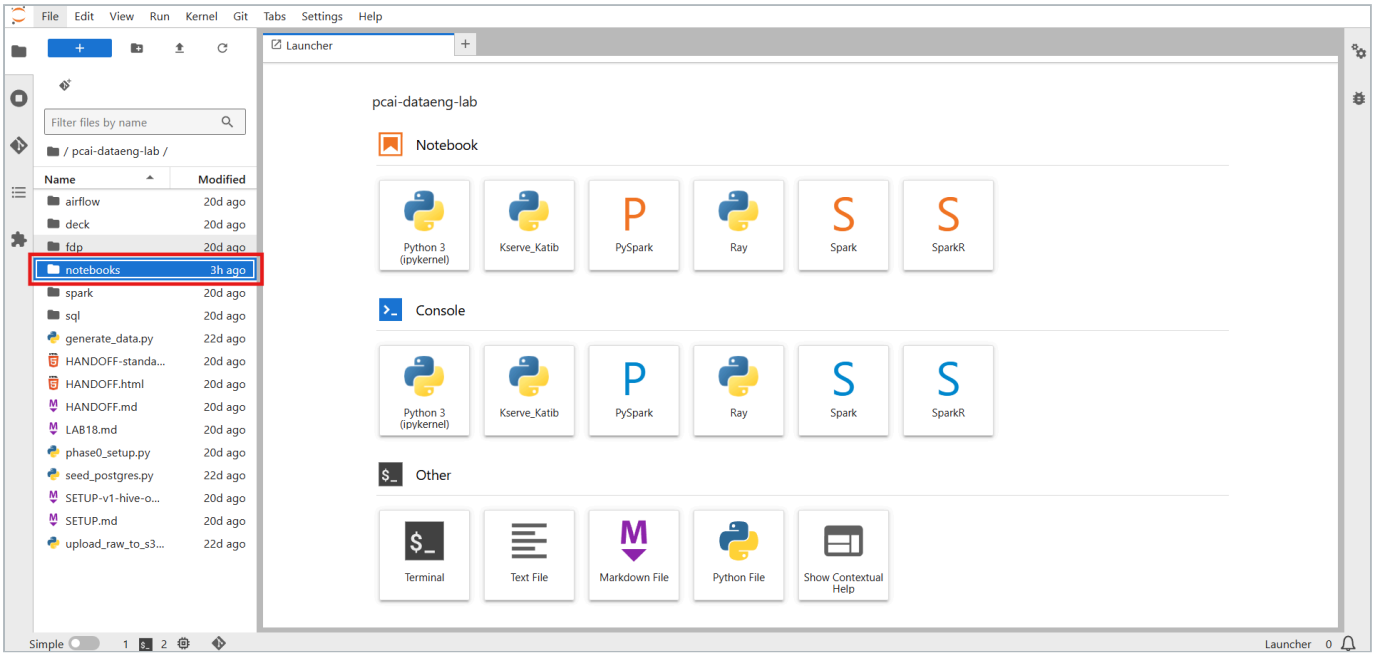

Figure 15. Inside pcai-dataeng-lab, open the notebooks folder.

The notebook opens in a new tab. Run the cells in order from the top: select a cell and click the **Run** button in the notebook toolbar, or press Shift+Enter. To run everything at once use **Kernel, Restart Kernel and Run All Cells**. Later cells use variables defined by earlier ones, so skipping ahead fails with a `NameError`.

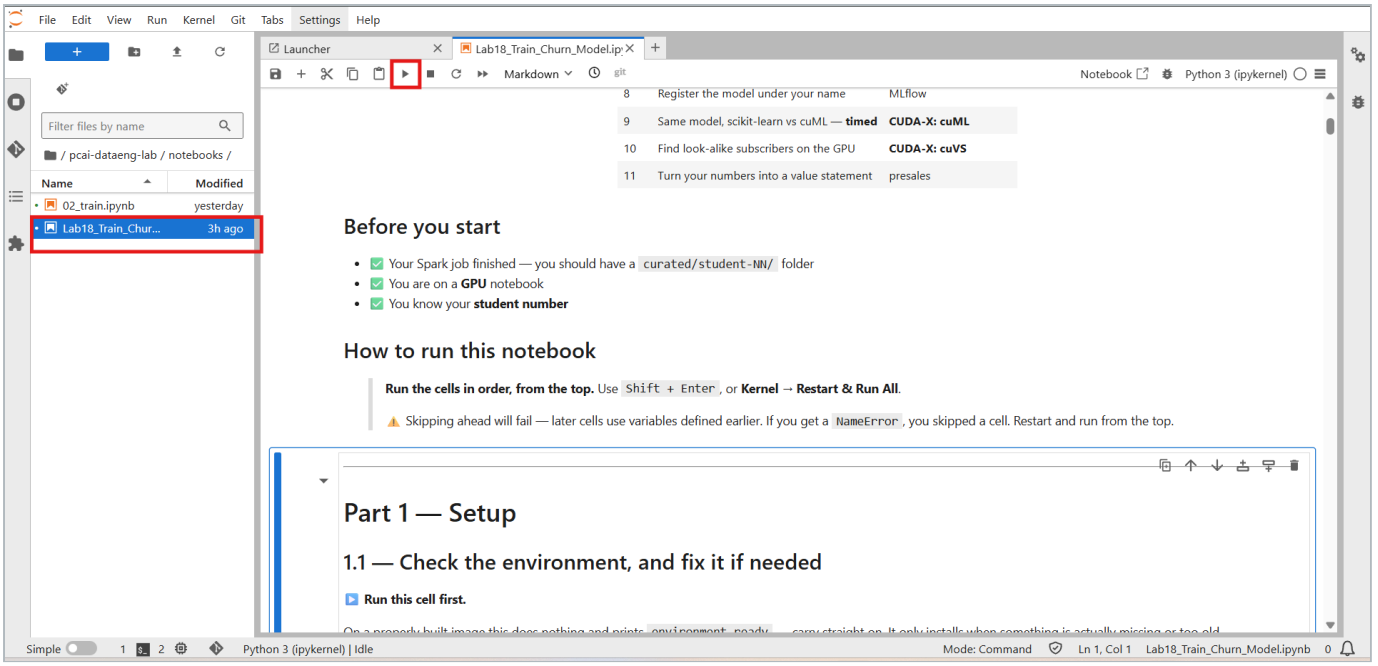

Figure 16. The lab notebook open in JupyterLab, with the Run button in the toolbar.

Part 1.1 - Check the environment

The first cell inspects the installed packages and prints `environment ready` when they match the required set. On a correctly built image it installs nothing. If it has to install, it says so and tells you to restart the kernel, which is unavoidable because NumPy is compiled into other packages. cuDF, cuML and cuVS are reported but never installed by the notebook. They must be baked into the image.

```
numpy 2.4.6 · pandas 3.0.3 · cuDF present · cuML · cuVS

environment ready — no restart needed, carry on to 1.2
```

Part 1.2 - Set your student number

This is the only cell you edit. Everything else, which folder you read, what your experiment is called, what your model is named, is built from this number.

1. In the code cell, find the line `STUDENT_ID = 0`.
2. Replace the `0` with your own student number, without leading zeros. Student 7 types `7`.
3. Run the cell.

Screenshot to be inserted: 09c_student_id.png

Figure 17. Part 1.2. Replace the value after STUDENT_ID with your own student number before running the cell.

```
import os

STUDENT_ID = 0                      # ←←← CHANGE THIS TO YOUR NUMBER

SID     = f'{STUDENT_ID:02d}'       # zero-padded: 0 -> '00', 7 -> '07'
LAB     = os.path.expanduser('~/shared/data-engineering-lab')
CURATED = f'{LAB}/curated/student-{SID}/watch_events'
CUTOFF  = '2026-05-31'              # we use the LAST 30 DAYS of history as features

print(f'You are student {SID}')
print(f'Reading from : {CURATED}')
print(f'Feature window: events on or after {CUTOFF}')

assert os.path.isdir(CURATED), (
    f'\nNo curated data at {CURATED}\n'
    f'Either your Spark job has not finished, or STUDENT_ID is wrong.')
print('\n▫ Your curated data is there — carry on.')
```

The cell prints your zero-padded number, the curated folder it will read, and the feature window, then confirms the folder exists.

```
You are student 07
Reading from : /home/<user>/shared/data-engineering-lab/curated/student-07/watch_events
Feature window: events on or after 2026-05-31

▫ Your curated data is there — carry on.
```

If the check fails, either the Spark job has not finished or the number does not match the one used in Stage 3.

Part 1.3 - GPU check and memory cap

The cell initialises the RAPIDS memory manager with a 1.5 GB pool cap and prints the GPU. Everything in the lab fits comfortably inside that cap, and on a shared card it is what protects the other users.

```
import subprocess

try:
    import rmm                                      # RAPIDS memory manager
    rmm.reinitialize(pool_allocator=True,
                     initial_pool_size='256MB',
                     maximum_pool_size='1500MB')  # ← the cap that protects everyone else
    import cudf
    HAS_GPU_DF = True
    print(f'▫ cuDF {cudf.__version__} available — GPU memory capped at 1.5 GB')
except Exception as e:
    HAS_GPU_DF, cudf = False, None
    print('⚠  cuDF not available — the notebook will use the CPU instead.')
    print('   (', str(e)[:70], ')')

print()
print(subprocess.run(['nvidia-smi','--query-gpu=name,memory.total,memory.used',
                      '--format=csv'], capture_output=True, text=True).stdout)
```

```
▫ cuDF 26.08.01 available — GPU memory capped at 1.5 GB

name, memory.total [MiB], memory.used [MiB]
NVIDIA L40S, 46068 MiB, 1234 MiB
```

Part 2 - What Spark produced

Only the last 30 day folders are read. The other 150 are never opened. That is partition pruning.

```
# Guard: this needs the Config cell in Part 1. Running cells out of order is the most
# common mistake, and a bare NameError does not explain it.
for _v in ('SID', 'CURATED', 'CUTOFF'):
    if _v not in globals():
        raise RuntimeError(f"'{_v}' is not defined — run Part 1 first "
                           "(Kernel -> Restart & Run All).")

import glob, datetime

cutoff   = datetime.date.fromisoformat(CUTOFF)
day_dirs = sorted(glob.glob(f'{CURATED}/dt=*'))
keep     = [d for d in day_dirs
            if datetime.date.fromisoformat(d.split('dt=')[1]) >= cutoff]
files    = [f for d in keep for f in glob.glob(f'{d}/*.parquet')]

print(f'day-folders Spark wrote   : {len(day_dirs)}')
print(f'day-folders we will read  : {len(keep)}   ← the other '
      f'{len(day_dirs)-len(keep)} are never opened')
print(f'parquet files to read     : {len(files)}')
```

```
day-folders Spark wrote   : 180
day-folders we will read  : 30   ← the other 150 are never opened
parquet files to read     : 30
```

Part 3 - The squash, CPU then GPU

The same function rolls every subscriber's last 30 days into one row: total minutes, number of sessions, completion rate and distinct titles. It runs first with pandas on the CPU, then with cuDF on the GPU. The function does not change. Only the library that read the data does.

```
import pandas as pd, time

COLS = ['subscriber_id', 'watch_minutes', 'completed', 'content_id']

def squash(df):
    """Many rows per person -> one row per person.
    This exact function runs on BOTH pandas and cuDF — same API, different hardware."""
    g = df.groupby('subscriber_id').agg(
        {'watch_minutes': ['sum', 'count'],
         'completed': 'mean',
         'content_id': 'nunique'})
    g.columns = ['minutes_30d', 'sessions_30d', 'completion_rate', 'distinct_titles_30d']
    return g.reset_index()

print('Reading with pandas (CPU) ...')
t0 = time.time()
pdf = pd.concat([pd.read_parquet(f, columns=COLS) for f in files], ignore_index=True)
t_read_cpu = time.time() - t0

print('Squashing with pandas (CPU) ...')
t0 = time.time()
usage_cpu = squash(pdf)
t_agg_cpu = time.time() - t0

print(f'\n  rows read       : {len(pdf):,}')
print(f'  people out      : {len(usage_cpu):,}')
print(f'  CPU read time   : {t_read_cpu:6.2f} s')
print(f'  CPU squash time : {t_agg_cpu:6.2f} s')
usage_cpu.head()
```

```
if HAS_GPU_DF:
    print('Reading with cuDF (GPU) ...')
    t0 = time.time()
    gdf = cudf.read_parquet(files, columns=COLS)
    t_read_gpu = time.time() - t0

    print('Squashing with cuDF (GPU) ...')
    t0 = time.time()
    usage_gpu = squash(gdf)          # ← the SAME function as the CPU cell
    t_agg_gpu = time.time() - t0

    print(f'\n  rows read       : {len(gdf):,}')
    print(f'  GPU read time   : {t_read_gpu:6.2f} s')
    print(f'  GPU squash time : {t_agg_gpu:6.2f} s')
    print('\n' + '=' * 52)
    print(f'  SQUASH SPEED-UP : {t_agg_cpu / max(t_agg_gpu, 1e-6):5.1f}x'
          f'   (CPU {t_agg_cpu:.2f}s  ->  GPU {t_agg_gpu:.2f}s)')
    print('=' * 52)
    usage = usage_gpu.to_pandas()
else:
    print('cuDF not available — using the CPU result from the previous cell.')
    usage = usage_cpu

print(f'\n{len(usage):,} people, one row each. This is HALF of the training data —')
print('it says what people DID, but not whether they cancelled.')
usage.head()
```

Expected: 3,286,532 rows read, 196,564 people out, and a squash speed-up line comparing the two timings. On the reference L40S run the CPU squash took 0.39 seconds and the GPU squash 0.03 seconds, a 12 times speed-up. The GPU read includes a one-time start-up cost, so the squash time is the honest comparison.

```
Reading with cuDF (GPU) ...
Squashing with cuDF (GPU) ...

  rows read       : 3,286,532
  GPU read time   :   0.38 s
  GPU squash time :   0.03 s

====================================================
  SQUASH SPEED-UP :  12.0x   (CPU 0.39s  ->  GPU 0.03s)
====================================================

196,564 people, one row each. This is HALF of the training data —
it says what people DID, but not whether they cancelled.
```

	subscriber_id	minutes_30d	sessions_30d	completion_rate	distinct_titles_30d
0	1	695.2	19	0.421053	19
1	2	439.9	15	0.466667	15
2	3	922.3	29	0.482759	29
3	4	110.9	5	0.800000	5
4	5	1190.7	36	0.666667	36

Figure 18. Part 3 output on the GPU. The same aggregation as the CPU cell, with the measured speed-up and the first rows of the squashed table.

Important

The squash produces 196,564 people, not 200,000. The remaining 3,436 subscribers watched nothing in the last 30 days and therefore have no row here. Part 5 is where they come back.

Part 4 - The label lives somewhere else

The cell reads the 200,000 subscriber rows from Postgres with a read-only account: plan, country, payment method, price, tickets, tenure and the churn label. Expected: 200,000 rows and a churn rate of 0.1190.

```
# psycopg2 is small and shares no dependencies with numpy/pandas/cuDF, so unlike
# cuDF it is safe to install inline — no kernel restart needed. The version pins
# stop pip from quietly downgrading numpy underneath cuDF.
try:
    import psycopg2
except ImportError:
    import sys, subprocess
    print('installing psycopg2-binary ...')
    subprocess.run([sys.executable, '-m', 'pip', 'install', '-q',
                    'psycopg2-binary', 'numpy>=2,<3', 'pandas>=3.0,<3.0.4'],
                   check=False)
    import psycopg2
    print('ok')


PG = dict(host='postgres-data-postgresql.postgres-data.svc.cluster.local',
          port=5432, dbname='app_db',
          user='admin', password='admin',      # read-only account
          connect_timeout=10)

conn = psycopg2.connect(**PG)
subs_pd = pd.read_sql('''
    SELECT subscriber_id, plan, country, payment_method, monthly_price,
           support_tickets_90d, tenure_months, churned_next_30d
...
```

Part 5 - Join the two halves on the GPU

Behaviour meets identity on `subscriber_id`. The join is a **left** join from subscribers with missing activity filled as zero, so the 3,436 silent subscribers are kept.

```
def to_gpu(pdf):
    """pandas -> cuDF. The function name moved between cuDF versions, so try both."""
    return cudf.from_pandas(pdf) if hasattr(cudf, 'from_pandas') else cudf.DataFrame(pdf)

FILL = {'minutes_30d': 0.0, 'sessions_30d': 0,
        'completion_rate': 0.0, 'distinct_titles_30d': 0}

if HAS_GPU_DF:
    feat_df = (to_gpu(subs_pd)
               .merge(to_gpu(usage), on='subscriber_id', how='left')
               .fillna(FILL)
               .to_pandas())
    where = 'GPU'
else:
    feat_df = (subs_pd.merge(usage, on='subscriber_id', how='left').fillna(FILL))
    where = 'CPU'

print(f'Training table ({where} join): {len(feat_df):,} rows x {feat_df.shape[1]} columns\n')

summary = (feat_df.groupby('churned_next_30d')
                  .agg(people=('subscriber_id', 'count'),
                       avg_minutes_30d=('minutes_30d', 'mean'),
                       avg_sessions=('sessions_30d', 'mean')).round(1))
summary.index = ['stayed (0)', 'churned (1)']
print(summary)
print('\n▫ People who cancelled watched far less. That gap IS the model.')
```

Expected: a 200,000 by 12 training table, and a two-row summary showing that people who churned watched about 205 minutes against about 577 for people who stayed. That gap is what the model will learn.

Part 5b - The validation checklist, executed

Before anything is trained, the cell runs nine checks that mirror the rows of the transformation and validation checklist: join completeness, key uniqueness, nulls, value ranges, the label base rate, the zero-activity group, and equality of the CPU and GPU aggregations. It stops the notebook if any check fails.

```
▫ PASS  [3.2] LEFT join kept every subscriber    200,000 in, 200,000 out
▫ PASS  [3.4] join key unique on both sides      subscriber_id unique
▫ PASS  [3.3] no nulls after fill                0 nulls
▫ PASS  [3.5] zero-activity group preserved      3,436 people watched nothing; 63% of them
churned
▫ PASS  [2.5] completion_rate within [0, 1]      max 1.000
▫ PASS  [2.5] counts non-negative                ok
▫ PASS  [0.5] label base rate near 0.12          0.1190
▫ PASS  [3.6] churners watch less than stayers   mean minutes {0: 577, 1: 205}
▫ PASS  [2.4] CPU and GPU squash identical       196,564 rows, same total minutes

ALL PASS — this table is ready to train on.
```

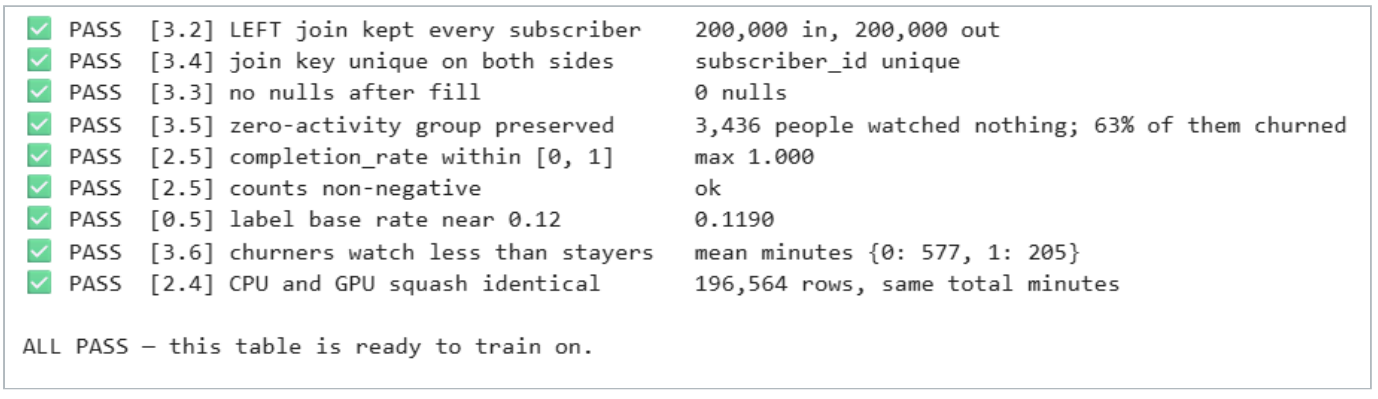

Figure 19. Part 5b. Every transformation validated before training.

The line for the zero-activity group is the most important number in the lab. Sixty-three percent of the people who watched nothing churned, against twelve percent overall. An inner join would have deleted them, and with them 2,172 churners, without any error.

Stage 5 - Train, explain, register

Part 6 - Train the model

Words become numbers by one-hot encoding plan, country and payment method, giving 19 features. A stratified split keeps 25 percent hidden. `device='cuda'` puts XGBoost on the GPU with no other change.

```
import xgboost as xgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

y = feat_df['churned_next_30d']
X = feat_df.drop(columns=['subscriber_id', 'churned_next_30d'])
X = pd.get_dummies(X, columns=['plan', 'country', 'payment_method'])  # words -> numbers

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

print(f'training on {len(X_tr):,} people, testing on {len(X_te):,} held-out people')
print(f'{X.shape[1]} features after one-hot encoding\n')

def make(device):
    return xgb.XGBClassifier(n_estimators=300, max_depth=6, learning_rate=0.1,
                             subsample=0.9, tree_method='hist', device=device,
                             eval_metric='auc')

try:
    model = make('cuda'); model.fit(X_tr, y_tr); dev = 'GPU'
except Exception as e:
    print('GPU training unavailable, falling back to CPU:', str(e)[:70])
    model = make('cpu');  model.fit(X_tr, y_tr); dev = 'CPU'

model.set_params(device='cpu')      # inference on host data; avoids a device-mismatch warning
auc = roc_auc_score(y_te, model.predict_proba(X_te)[:, 1])

print(f'\n{"=" * 46}')
print(f'  TEST AUC: {auc:.4f}     ({dev} training)')
print('=' * 46)
print('\n▫ Write this number down — it goes on the leaderboard.')
```

Expected: a test AUC near 0.87. AUC is the probability that a randomly chosen churner scores higher than a randomly chosen non-churner. A value of 0.5 is a coin flip, and anything near 1.0 should make you suspect a leak.

Part 7 - Which columns mattered

```
imp = (pd.Series(model.feature_importances_, index=X.columns)
         .sort_values(ascending=False))

print('TOP 8 FEATURES')
print(imp.head(8).to_string())

noise = imp[[c for c in imp.index if c.startswith('country_')]]
real  = imp[[c for c in imp.index if not c.startswith('country_')]]

print('\nTHE PLANTED NOISE (country)')
print(noise.to_string())

print('\n' + '=' * 58)
print(f'  noise columns : spread {noise.max()-noise.min():.4f}  '
      f'(from {noise.min():.4f} to {noise.max():.4f})')
print(f'  real  columns : spread {real.max()-real.min():.4f}  '
      f'(from {real.min():.4f} to {real.max():.4f})')
print('=' * 58)
print('Real features disagree with each other. Noise features do not.')
```

Two things to look for. Columns that measure the same underlying thing share the credit, so `minutes_30d` ranks low even though churners watch a third as much; importance shows what the model used, not what the world contains. And the six `country_*` columns land within 0.002 of each other while the real features differ by ten times. Real features disagree with each other. Noise features are indistinguishable.

```
TOP 8 FEATURES
sessions_30d              0.432158
distinct_titles_30d       0.284886
support_tickets_90d       0.047247
monthly_price             0.028597
payment_method_prepaid    0.027612
tenure_months             0.022455
minutes_30d               0.014820
country_GB                0.014662

THE PLANTED NOISE (country)
country_GB    0.014662
country_JP    0.014520
country_BR    0.014121
country_DE    0.013542
country_IN    0.013540
country_US    0.012586

  noise columns : spread 0.0021  (from 0.0126 to 0.0147)
  real  columns : spread 0.4322  (from 0.0000 to 0.4322)
Real features disagree with each other. Noise features do not.
```

```
TOP 8 FEATURES
sessions_30d              0.432158
distinct_titles_30d       0.284886
support_tickets_90d       0.047247
monthly_price             0.028597
payment_method_prepaid    0.027612
tenure_months             0.022455
minutes_30d               0.014820
country_GB                0.014662

THE PLANTED NOISE (country)
country_GB    0.014662
country_JP    0.014520
country_BR    0.014121
country_DE    0.013542
country_IN    0.013540
country_US    0.012586

============================================================
  noise columns : spread 0.0021  (from 0.0126 to 0.0147)
  real  columns : spread 0.4322  (from 0.0000 to 0.4322)
============================================================
Real features disagree with each other. Noise features do not.
```

Figure 20. Part 7 output. The feature importance with the noise fingerprint: six country columns within 0.002 of each other.

The exact values vary slightly from run to run because the two engagement features share the credit differently each time, but the shape does not: two engagement features at the top, tickets and plan next, and the six countries bunched together at the bottom.

Part 8 - Register the model in MLflow

The cell logs the four hyperparameters, the AUC, a set of tags and the model itself, and registers it as `student-NN-churn`. The platform token is read automatically from the notebook environment.

```
import mlflow

try:
    uri = os.environ.get('MLFLOW_TRACKING_URI',
                         'http://mlflow.mlflow.svc.cluster.local:5000')
    tok = '/etc/secrets/ezua/.auth_token'
    if os.path.exists(tok):
        os.environ['MLFLOW_TRACKING_TOKEN'] = open(tok).read().strip()
    mlflow.set_tracking_uri(uri)
    mlflow.set_experiment(f'student-{SID}-churn')

    with mlflow.start_run(run_name=f'student-{SID}') as run:
        mlflow.log_params({'n_estimators': 300, 'max_depth': 6,
                           'learning_rate': 0.1, 'subsample': 0.9})
        mlflow.log_metric('auc', auc)
        mlflow.set_tags({'student_id': SID, 'dataset': 'streaming-churn',
                         'rows_trained_on': len(X_tr)})
        mlflow.xgboost.log_model(model, 'model',
...
```

```
Successfully registered model 'student-07-churn'.
▫ Registered as: student-07-churn
   AUC 0.8712
Created version '1' of model 'student-07-churn'.
```

```
/opt/conda/lib/python3.11/site-packages/xgboost/sklearn.py:1116: UserWarning: [10:24:35] WARNING: /__w/xgboost/xgboost/src/c_api/c_api.cc:1573: Saving
model in the UBJSON format as default.  You can use a file extension: `json` or `ubj` to choose between formats.
  self.get_booster().save_model(fname)
2026/09/08 10:24:39 WARNING mlflow.models.model: Model logged without a signature and input example. Please set `input_example` parameter when logging
the model to auto infer the model signature.
Registered model 'student-00-churn' already exists. Creating a new version of this model...
2026/09/08 10:24:39 INFO mlflow.store.model_registry.abstract_store: Waiting up to 300 seconds for model version to finish creation. Model name: stude
nt-00-churn, version 2
Created version '2' of model 'student-00-churn'.
🏃 View run student-00 at: http://mlflow.mlflow.svc.cluster.local:5000/#/experiments/41/runs/c58f57ea83b24a1c97c7384f755eda2e
🧪 View experiment at: http://mlflow.mlflow.svc.cluster.local:5000/#/experiments/41

✅ Registered as: student-00-churn
   AUC 0.8682
```

Figure 21. Part 8 output. The warnings are expected. Running the cell again creates the next version of the same registered model.

Step 5.1 - Find your run in MLflow

1. In the left navigation choose **Tools & Frameworks**, find the **MLflow** tile and click **Open**. MLflow opens in a new tab.
2. On the **Experiments** page, type your student number in the search box on the left, for example `student-07`, and click the experiment `student-NN-churn`.
3. Click the run named `student-NN`. The **Overview** tab shows who created it, its status, its tags and the registered model with its version.
4. Click the **Model metrics** tab. The `auc` chart shows the held-out score that Part 6 printed.

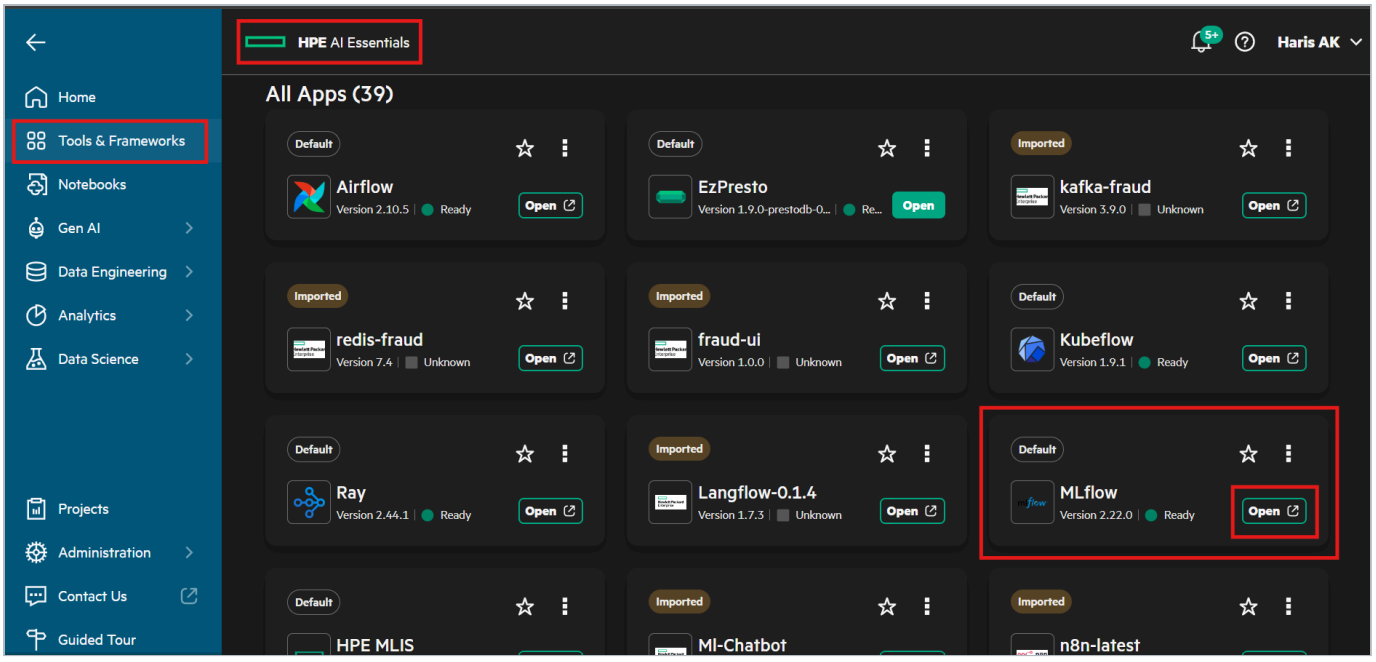

Figure 22. The MLflow tile under Tools & Frameworks.

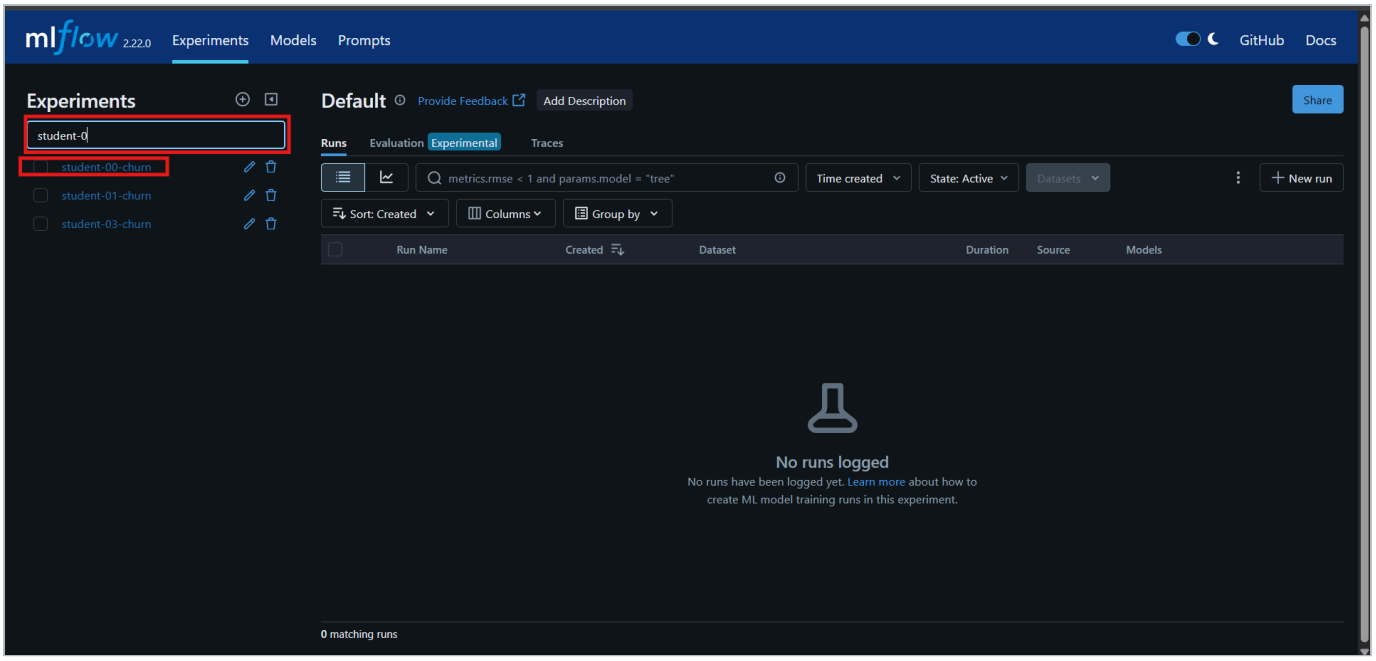

Figure 23. MLflow Experiments. Search for your student number and open your experiment.

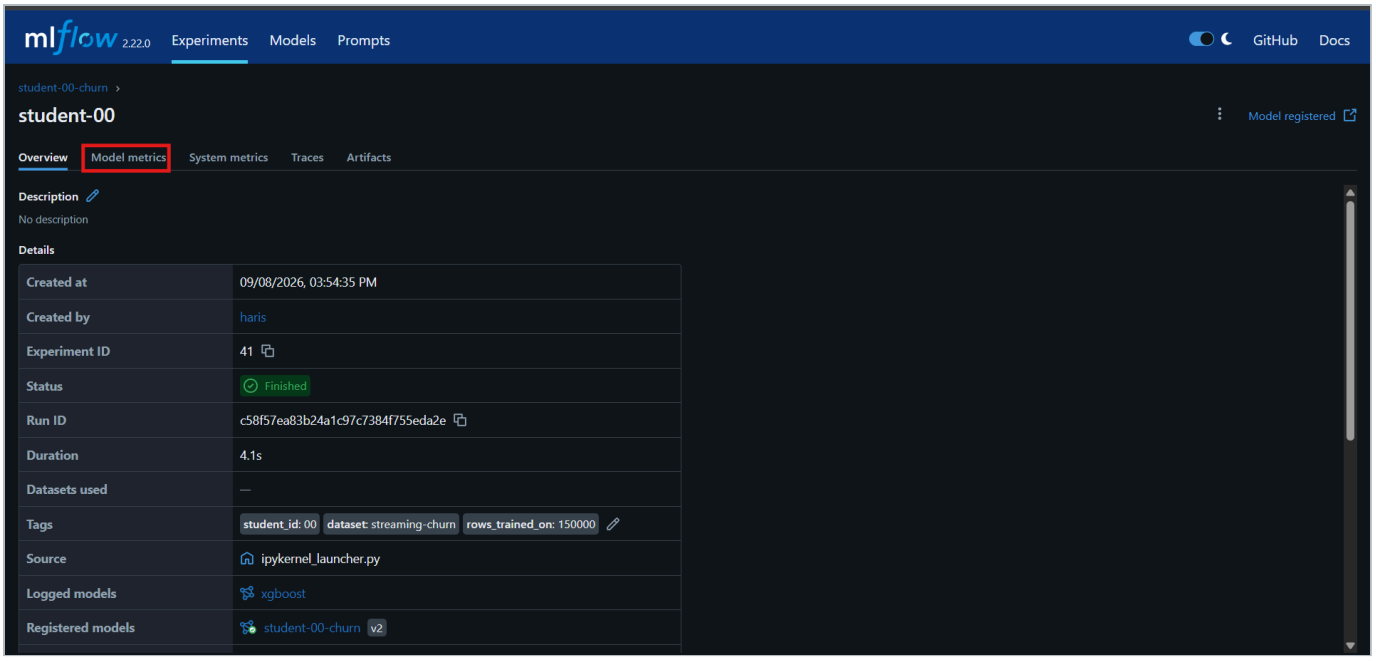

Figure 24. The run Overview. Tags, source, the logged model and the registered model version are all recorded.

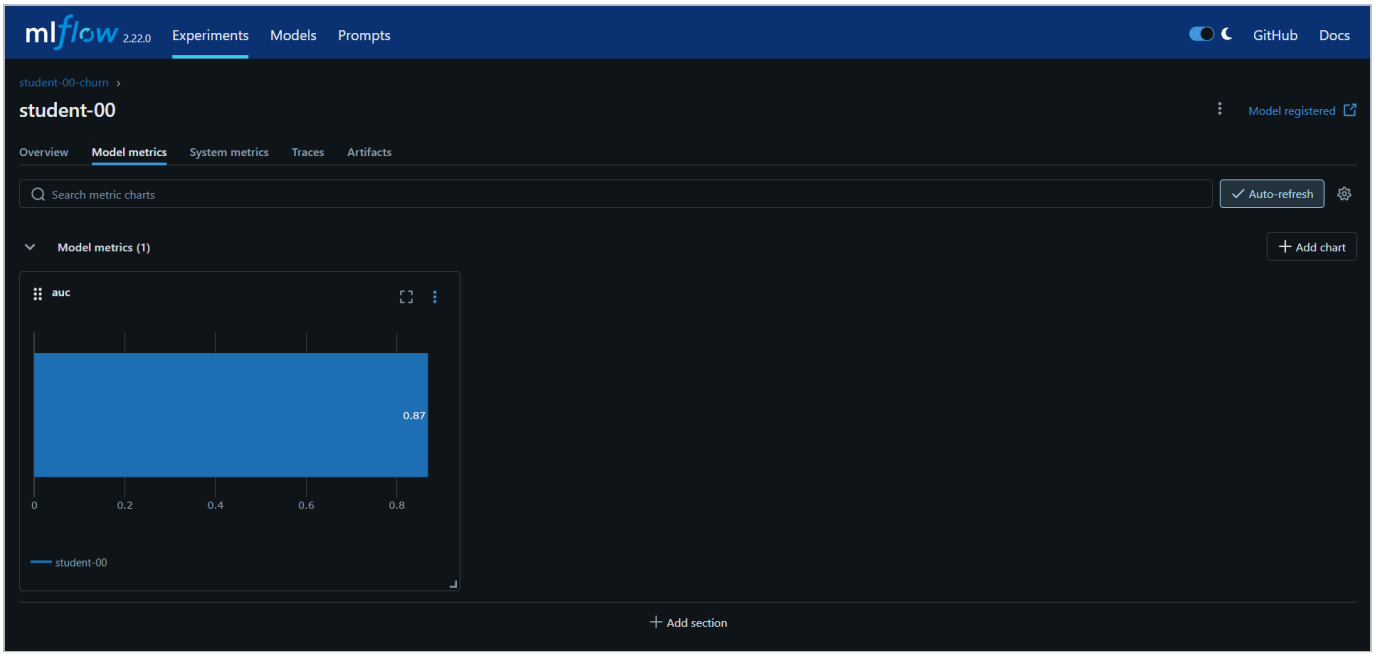

Figure 25. The Model metrics tab with the auc value.

Compare your AUC with the other participants by opening their experiments the same way. With one dominant signal, hyperparameters move it by a fraction of a percent.

Note

To create a second version, change `max_depth`, `n_estimators` or `learning_rate` in Part 6 and rerun Parts 6 to 8. The registry shows version 2 beside version 1. Do not expect large gains: better features beat better hyperparameters almost always.

Stage 6 - Measure CUDA-X and write the value

Part 9 - The same model on cuML

cuML is a GPU implementation of the scikit-learn API. The cell trains a RandomForest with identical parameters twice, scikit-learn on the CPU cores the notebook has and cuML on the GPU, checks that the two AUCs agree, and

only then compares the clock. A fast wrong answer is worth nothing, so accuracy is checked first.

```
import os, time
import numpy as np
from sklearn.ensemble import RandomForestClassifier

def cpu_cores_allowed():
    """CPU cores this NOTEBOOK may use. os.cpu_count() reports the whole node (e.g. 128),
    but a Kubernetes pod is capped by its cgroup quota -- that cap is the honest number
    to compare a GPU against, and the right thread count for scikit-learn."""
    for path in ('/sys/fs/cgroup/cpu.max',):                    # cgroup v2
        try:
            quota, period = open(path).read().split()
            if quota != 'max': return max(1, int(int(quota) / int(period)))
        except Exception: pass
    try:                                                          # cgroup v1
        q = int(open('/sys/fs/cgroup/cpu/cpu.cfs_quota_us').read())
        p = int(open('/sys/fs/cgroup/cpu/cpu.cfs_period_us').read())
        if q > 0: return max(1, int(q / p))
    except Exception: pass
    return os.cpu_count()

X_tr32, X_te32 = X_tr.astype('float32'), X_te.astype('float32')
y_tr32 = y_tr.astype('int32')
RF = dict(n_estimators=200, max_depth=14, random_state=0)
n_cores = cpu_cores_allowed()

t0 = time.time()
sk_rf = RandomForestClassifier(n_jobs=n_cores, **RF).fit(X_tr32, y_tr32)
t_rf_cpu = time.time() - t0
auc_rf_cpu = roc_auc_score(y_te, sk_rf.predict_proba(X_te32)[:, 1])
print(f'scikit-learn RandomForest : {t_rf_cpu:6.2f} s   AUC {auc_rf_cpu:.4f}   ({n_cores} CPU
cores)')

try:
    from cuml.ensemble import RandomForestClassifier as cuRandomForest
    # one tiny warm-up fit so the timing below is the model, not GPU start-up
    cuRandomForest(n_estimators=2, max_depth=2).fit(X_tr32.values[:1000], y_tr32.values[:1000])
    t0 = time.time()
    cu_rf = cuRandomForest(**RF).fit(X_tr32.values, y_tr32.values)
    t_rf_gpu = time.time() - t0
    auc_rf_gpu = roc_auc_score(y_te, np.asarray(cu_rf.predict_proba(X_te32.values))[:, 1])
    print(f'cuML RandomForest         : {t_rf_gpu:6.2f} s   AUC {auc_rf_gpu:.4f}   (GPU)')
    print('\n' + '=' * 60)
    print(f'  AUC gap  : {abs(auc_rf_cpu - auc_rf_gpu):.4f}   (same model within noise)')
    print(f'  SPEED-UP : {t_rf_cpu / max(t_rf_gpu, 1e-6):.1f}x   ({n_cores} CPU cores  ->  1
GPU)')
    print('=' * 60)
    HAS_CUML_RESULT = True
except Exception as e:
    print('cuML not available — CPU result only. (', str(e)[:70], ')')
    t_rf_gpu, auc_rf_gpu, HAS_CUML_RESULT = None, None, False
```

```
scikit-learn RandomForest :   4.02 s   AUC 0.8668   (4 CPU cores)
cuML RandomForest         :   1.69 s   AUC 0.8659   (GPU)

  AUC gap  : 0.0009   (same model within noise)
  SPEED-UP : 2.4x   (4 CPU cores  ->  1 GPU)
```

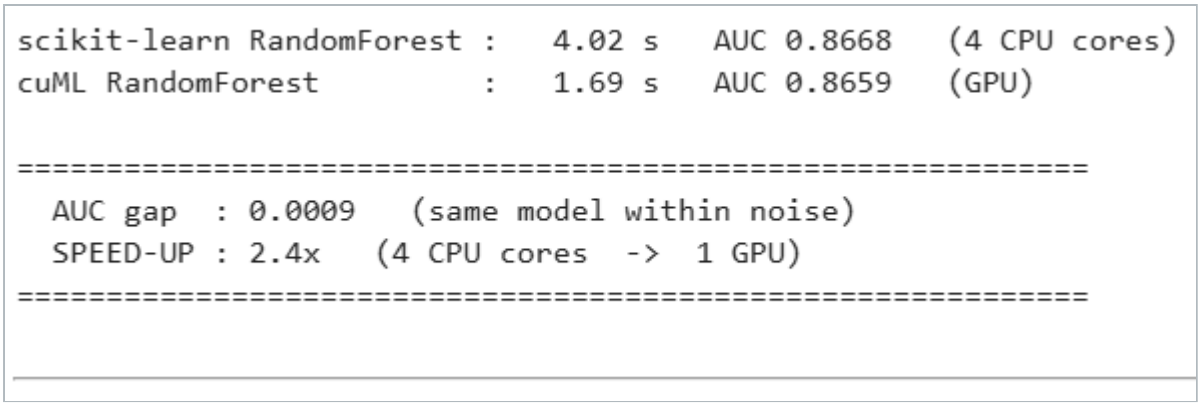

Figure 26. Part 9 output on the reference L40S run. Accuracy agrees, then the clock is compared against the four CPU cores of the notebook.

Expected: two AUCs within 0.001 of each other, and a speed-up line that names the CPU core count it was measured against. That named comparison is the number you may quote. Vendor headline figures are ceilings measured on chosen workloads.

Part 10 - Look-alike search with cuVS

A different question a retention team asks: who looks like the people who left? Every subscriber becomes a point in 19-dimensional space, and for each held-out subscriber the cell finds the ten most similar subscribers in the training set. The share of those neighbours that churned is a model-free churn score. The search runs with scikit-learn brute force on the CPU and cuVS brute force on the GPU, and recall between the two is checked before timing.

```
import time
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors

K = 10
scaler = StandardScaler().fit(X_tr32)
V_tr = np.ascontiguousarray(scaler.transform(X_tr32), dtype='float32')   # 150k vectors
V_te = np.ascontiguousarray(scaler.transform(X_te32), dtype='float32')   #  50k queries
y_tr_arr = y_tr.values

t0 = time.time()
nn = NearestNeighbors(n_neighbors=K, algorithm='brute', n_jobs=n_cores).fit(V_tr)
_, nbr_cpu = nn.kneighbors(V_te)
t_knn_cpu = time.time() - t0
score_cpu = y_tr_arr[nbr_cpu].mean(axis=1)            # share of churned neighbours
auc_knn_cpu = roc_auc_score(y_te, score_cpu)
print(f'scikit-learn brute force : {t_knn_cpu:6.2f} s   '
      f'{len(V_te):,} queries x {len(V_tr):,} vectors x {V_tr.shape[1]} dims   ({n_cores} CPU
cores)')

try:
    import cupy as cp
    from cuvs.neighbors import brute_force
    # NOTE: a brute-force index REFERENCES the GPU array it was built on; it does not
    # copy it. Keep that array in a variable until the search is done, or the search
    # reads freed memory and returns garbage (recall drops to ~0 -- we hit this).
    warm_v, warm_q = cp.asarray(V_tr[:1000]), cp.asarray(V_te[:10])
    brute_force.search(brute_force.build(warm_v), warm_q, K)            # warm-up
    t0 = time.time()
    V_tr_gpu, V_te_gpu = cp.asarray(V_tr), cp.asarray(V_te)
    index = brute_force.build(V_tr_gpu, metric='sqeuclidean')
    _, nbr_gpu = brute_force.search(index, V_te_gpu, K)
    nbr_gpu = cp.asarray(nbr_gpu).get()
    t_knn_gpu = time.time() - t0
    recall = np.mean([len(set(a) & set(b)) / K for a, b in zip(nbr_cpu, nbr_gpu)])
    score_gpu = y_tr_arr[nbr_gpu].mean(axis=1)
    auc_knn_gpu = roc_auc_score(y_te, score_gpu)
    print(f'cuVS brute force         : {t_knn_gpu:6.2f} s   (GPU)')
    print('\n' + '=' * 60)
    print(f'  recall@{K} vs CPU : {recall:.4f}   (same neighbours)')
    print(f'  SPEED-UP        : {t_knn_cpu / max(t_knn_gpu, 1e-6):.1f}x')
    print('=' * 60)
    HAS_CUVS_RESULT = True
except Exception as e:
    print('cuVS not available — CPU result only. (', str(e)[:70], ')')
    t_knn_gpu, recall, HAS_CUVS_RESULT = None, None, False

print(f'\nlook-alike churn score AUC : {auc_knn_cpu:.4f}   (no model was trained for this)')
print(f'trained XGBoost AUC        : {auc:.4f}')
print('\n▫ Similarity alone gets most of the way. The model earns the rest — and the model')
print('   is what you register, version and serve. Similarity is what you use in a meeting.')
```

```
scikit-learn brute force :   4.03 s   50,000 queries x 150,000 vectors x 19 dims   (4 CPU cores)
cuVS brute force         :   0.05 s   (GPU)

  recall@10 vs CPU : 0.9996   (same neighbours)
  SPEED-UP        : 85.2x

look-alike churn score AUC : 0.8128   (no model was trained for this)
trained XGBoost AUC        : 0.8682
```

```
scikit-learn brute force :   4.03 s   50,000 queries x 150,000 vectors x 19 dims   (4 CPU cores)
cuVS brute force         :   0.05 s   (GPU)

============================================================
  recall@10 vs CPU : 0.9996   (same neighbours)
  SPEED-UP         : 85.2x
============================================================

look-alike churn score AUC : 0.8128   (no model was trained for this)
trained XGBoost AUC        : 0.8682

👉 Similarity alone gets most of the way. The model earns the rest — and the model
   is what you register, version and serve. Similarity is what you use in a meeting.
```

Figure 27. Part 10 output on the reference L40S run. The GPU found the same neighbours, 85 times faster than four CPU cores.

Similarity alone gets most of the way, and the model earns the rest. The model is what you register, version and serve. Similarity is what you use in a meeting. Exact brute-force search is the right tool at 150,000 vectors of 19 dimensions. cuVS's approximate indexes such as CAGRA exist for millions of embedding-sized vectors, and at this shape they are slower and less accurate.

Important

A cuVS brute-force index references the GPU array it was built on without copying it. The cell keeps that array in a variable until the search completes. If the array is released early the search reads freed memory and recall silently drops to zero.

Part 11 - From your numbers to a value statement

The final cell collects every number you measured into one table. It is the raw material for the last deliverable.

```
YOUR MEASURED RESULTS  (student 07)

  Columnar storage (Spark → Parquet)  849 MB → 232 MB CSV→Parquet             3.66x smaller
  Partition pruning                   180 day folders written, 30 read        83% of files never
opened
  Absence as signal (LEFT join)       3,436 zero-activity subscribers kept    63% of them
churned vs 12% overall
  cuDF squash (20M → 200k)            CPU 0.39 s  GPU 0.03 s                  12.0x
  cuML RandomForest                   CPU 4.02 s  GPU 1.69 s  (4 cores)       2.4x
  cuVS look-alike search              CPU 4.03 s  GPU 0.05 s                  85.2x
  Model quality                       XGBoost AUC 0.8682; look-alike AUC 0.8128
  Reproducibility                     student-07-churn registered in MLflow   params + metric +
model, versioned
```

Write three value statements from your own run, each in three lines: the technical result, what it means for the work, and why the customer cares. Say only the third line to a customer, and have the first ready when they ask compared with what. At least one statement must be about a data-engineering result such as storage, federation or validation, not acceleration.

Deliverables

Three documents accompany the notebook in the lab folder. Complete each one with the numbers from your own run.

Table 8. Written deliverables

Deliverable	File	What you do with it
Transformation and validation checklist	`TRANSFORMATION_VALIDATION_CHECKLIST.md`	Thirty rows across six stages. Part 5b executes the notebook-side rows. Tick the Spark and EzPresto rows from your Stage 2 and 3 outputs.
Data-readiness explanation	`DATA_READINESS.md`	A one-page customer-facing template. Fill every bracketed value from your run and write the readiness verdict.
Presales value statement	`PRESALES_VALUE_STATEMENT.md`	Worked examples of result, meaning, value for each stage, and the list of things you must not say. Write your three statements.

Measured results

The values below come from validated runs of this lab on the platform described in the environment section. Every number is a measurement, and each cell of the notebook prints the corresponding value for your own run.

Table 9. Reference measurements

Measure	Value	Where you see it
Raw events	20,010,929 rows, 180 files, 849 MB	Stage 1, Spark driver log
Subscribers	200,000 rows, churn rate 0.1190	Stage 2, Part 4
Curated Parquet	20,010,929 rows, 180 partitions, 232 MB, 3.66x smaller	Stage 3
Spark job wall clock	About 1 min 50 s; 2 min 30 s trigger to success through Airflow	Stage 3
Squash	196,564 people from 3,286,532 events in the 30-day window; CPU 0.39 s, GPU 0.03 s, 12x	Part 3
Zero-activity subscribers	3,436 kept by the left join; 63% of them churned	Part 5b
Features	19 after one-hot encoding	Part 6
Test AUC, XGBoost	0.87 (0.868 to 0.871 across reference runs)	Part 6
Country importance spread	0.002, against 0.43 for real features	Part 7
cuML versus scikit-learn	AUC gap 0.0009; 4.02 s on 4 CPU cores versus 1.69 s on the L40S, 2.4x	Part 9
cuVS versus scikit-learn	Recall at 10 of 0.9996; 4.03 s on 4 CPU cores versus 0.05 s on the L40S, 85x	Part 10
Look-alike score AUC	0.813 with no trained model	Part 10

Reading the numbers honestly

- The Spark job changes the format, not the row count. Quote the 3.66x as a storage and read-time result, not as data reduction.
- Every speed-up depends on the CPU it was compared against. The cells print the core count. Quote both sides or neither.
- The cuDF read includes GPU start-up. The squash timing is the fair comparison.
- A held-out AUC close to 1.0 would indicate leakage, not a good model. The value near 0.87 is expected and is the design point of the dataset.
- Country and device carry no signal by construction. Do not attribute any business meaning to their importance values.
- The look-alike score is model-free and reaches 0.81. The trained model reaches 0.87. The difference is what training buys.
- The Part 10 speed-up is the largest in the lab because brute-force nearest-neighbour search is almost pure arithmetic, exactly the shape a GPU is built for. Do not generalise it to the other stages.

Reference

Platform locations

Table 10. Where things are

Item	Location
Platform home	`https://home.ai-application.pcai1.genai1.hou/`
Data sources and Query Editor	Data Engineering menu
Airflow	Tools & Frameworks, Airflow tile, Open
Spark applications	Analytics, Spark Applications
Notebooks	Notebooks, with your project chosen in the Project selector
MLflow	`https://mlflow.ai-application.pcai1.genai1.hou/`
Raw data, notebook path	`~/shared/data-engineering-lab/raw/watch_events/dt=*/events.csv`
Raw data, Spark path	`file:///mounts/shared-volume/shared/data-engineering-lab/raw/watch_events`
Curated data, notebook path	`~/shared/data-engineering-lab/curated/student-NN/watch_events/`, or shared, data-engineering-lab, curated in the file browser
The notebook file	`pcai-dataeng-lab/notebooks/Lab18_Train_Churn_Model.ipynb` in the file browser
Postgres, from the notebook	`postgres-data-postgresql.postgres-data.svc.cluster.local:5432/app_db`
EzPresto catalog	`dataengineeringlab.public.subscribers`

Artifacts you create

Table 11. Artifacts and where to find them

Artifact	Name	Where it lives
Spark application	`curate-student-NN`	Spark Applications list, and the Airflow Grid view
Curated Parquet	`curated/student-NN/watch_events/dt=*/`	Shared data volume
MLflow experiment	`student-NN-churn`	MLflow, Experiments
Registered model	`student-NN-churn`, version 1	MLflow, Models
Value statements and readiness page	Your copies of the deliverable documents	Your home directory

Troubleshooting

Table 12. Symptoms, causes and resolutions

Symptom	Cause and resolution
`NameError` in a notebook cell	A cell was skipped. Use Kernel, Restart and Run All.
Part 1.2 reports no curated data	The Spark job has not finished or the student number is wrong. Check the Airflow Grid view or the Spark Applications list.
Part 1.1 installs packages and asks for a restart	The image lacks part of the required set. Restart the kernel once and run from the top. If it asks again, the kernel imports a different package tree than pip installed; report it.
Part 3 fails with a dimensions error in pandas	numpy and pandas are mismatched, usually because the kernel was not restarted after an install. Restart and run from the top.
Part 8 reports 401 and a token error from MLflow	The notebook server is not in your own namespace, so the platform did not provision your MLflow token. Create the server in your own namespace.
EzPresto rejects a query	Remove the trailing semicolon and run one statement at a time.
EzPresto returns exactly 1,000 rows	The worksheet preview limit. Aggregates are unaffected.
Airflow trigger does nothing	The DAG is paused. Turn the toggle on and trigger again.
Airflow task fails in one second	The Student number field was left empty. Trigger again with it filled in.
Part 10 recall near zero	The GPU array behind the cuVS index was released before the search. The shipped cell keeps it in a variable; do not remove that line.
Part 9 shows the GPU slower than the CPU	The notebook has many CPU cores, or the first cuML call absorbed initialisation. The cell warms up first; rerun it and read the printed core count.

Further reading

- NVIDIA CUDA-X for data science: `https://developer.nvidia.com/topics/ai/data-science/cuda-x-for-data-science`
- RAPIDS documentation for cuDF, cuML and cuVS: `https://docs.rapids.ai/`
- Apache Spark SQL, DataFrames and Parquet: `https://spark.apache.org/docs/latest/sql-programming-guide.html`
- MLflow tracking and model registry: `https://mlflow.org/docs/latest/`
- Apache Airflow concepts, DAGs and triggering with configuration: `https://airflow.apache.org/docs/`
- HPE Private Cloud AI and AI Essentials documentation: `https://www.hpe.com/us/en/private-cloud-ai.html`